

哈爾濱工業大學（深圳）

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

操作系统 (Operating System)

第六章：内存管理

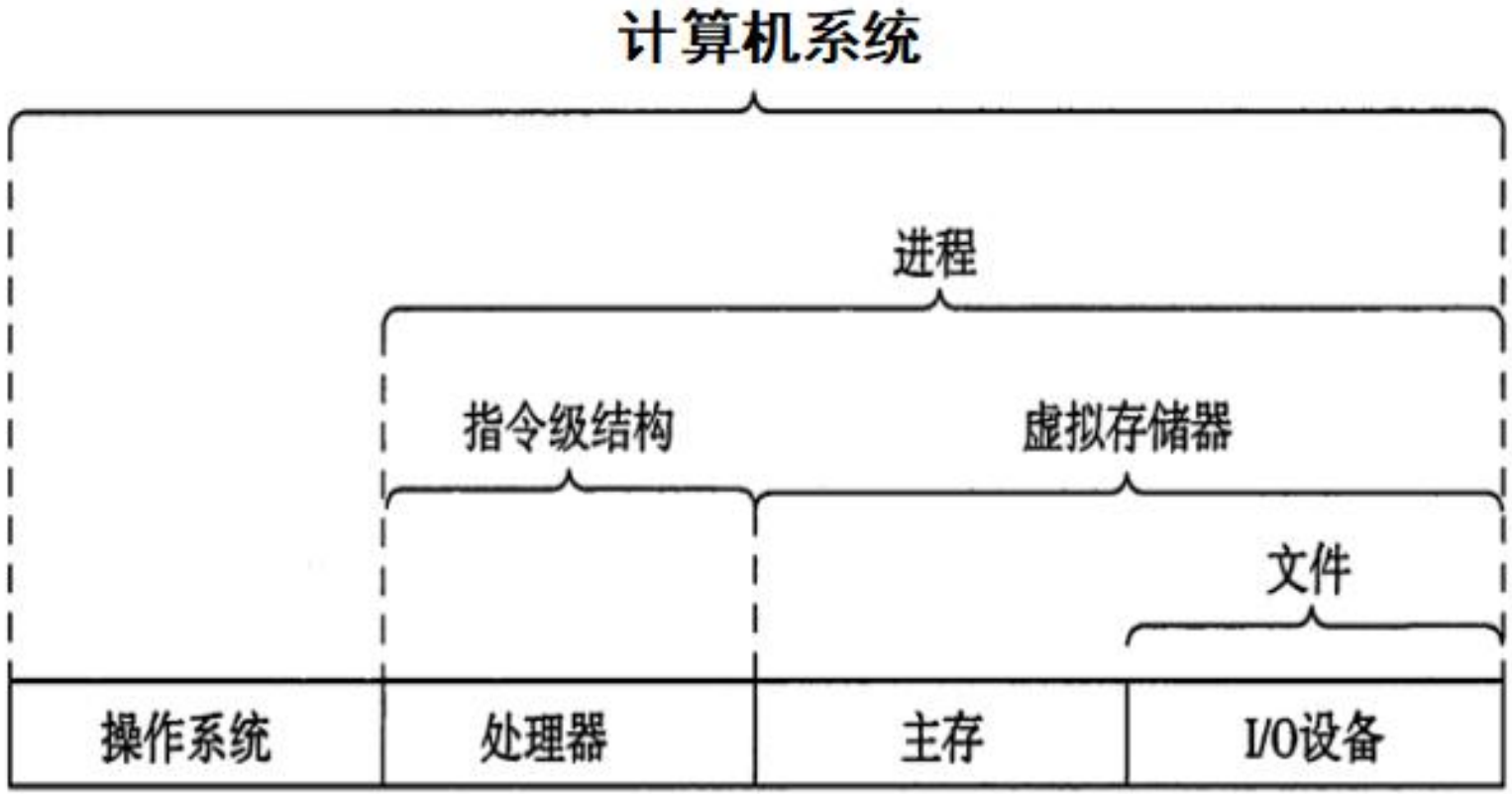
夏文 教授

哈尔滨工业大学（深圳）

2026年秋季

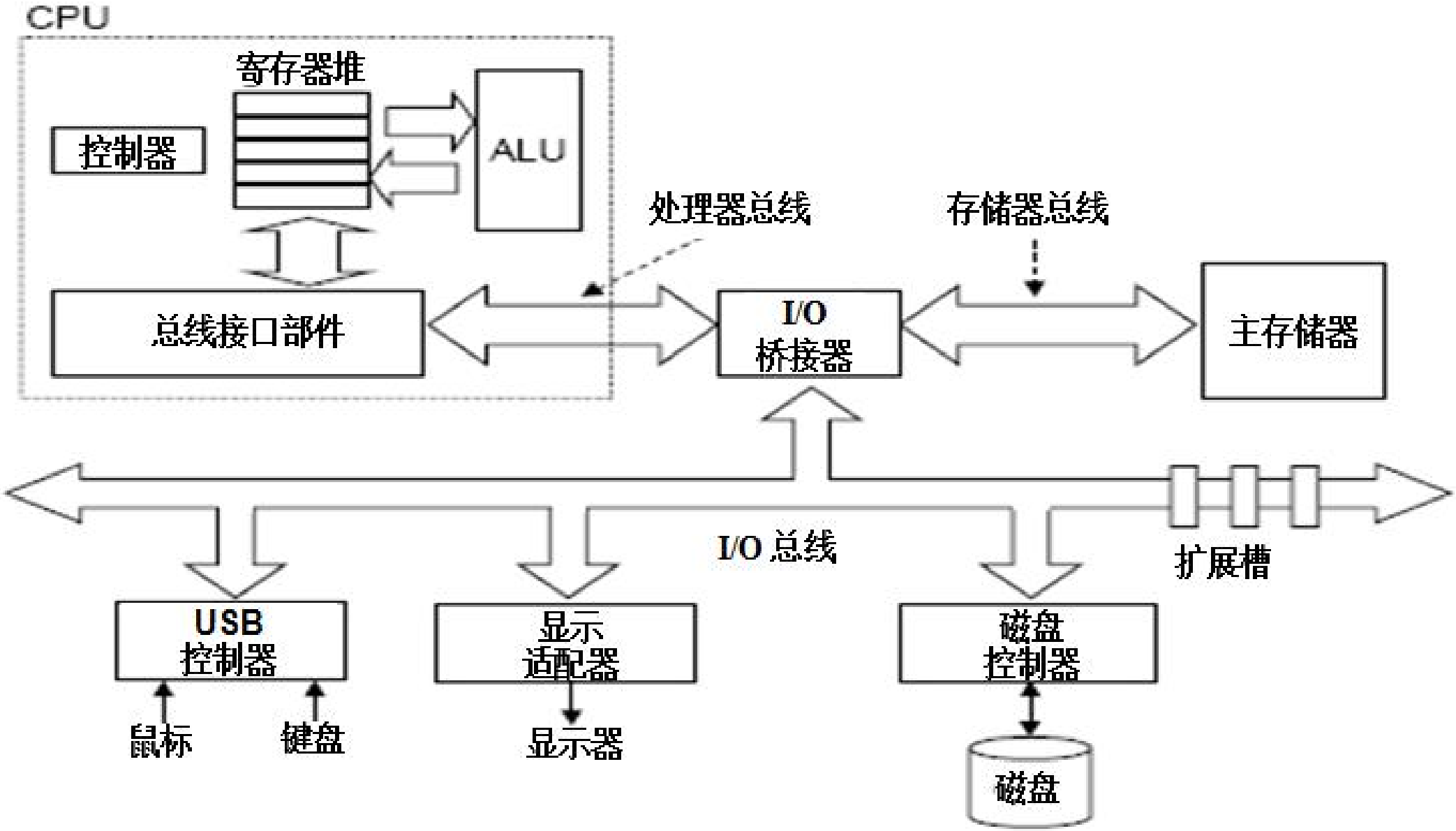
Email：xiawen@hit.edu.cn

计算机系统的抽象/虚拟化



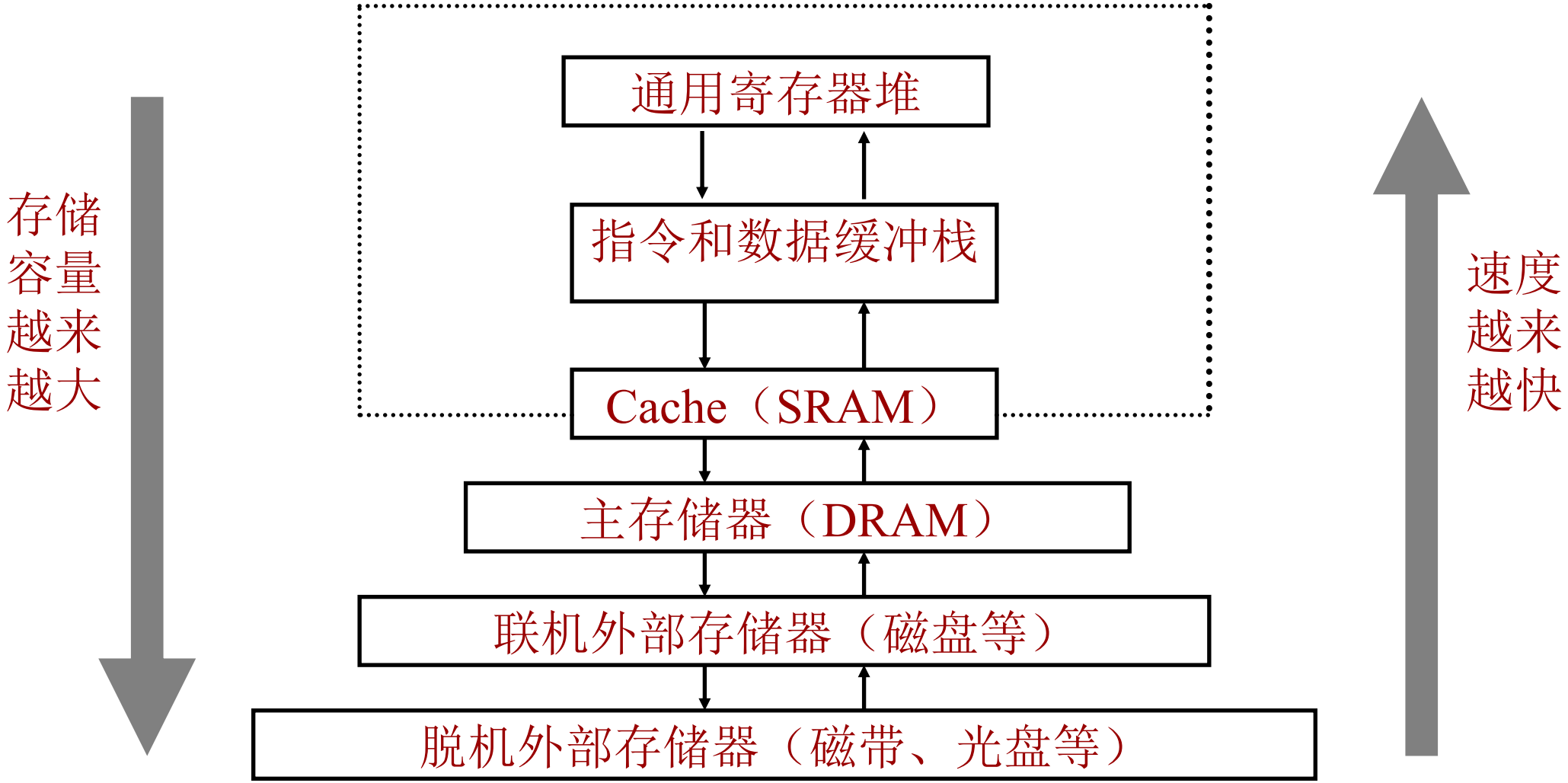
1、导论与操作系统结构
2、进程与线程
3、并发与同步
4、处理器调度
5、死锁
6、内存管理
7、虚拟内存
8、I/O与存储
9、文件及文件系统

计算机的体系结构


CPU
寄存器堆
控制器
ALU
处理器总线
存储器总线
总线接口部件
I/O
桥接器
主存储器
I/O 总线
扩展槽
USB
控制器
显示
适配器
磁盘
控制器
鼠标
键盘
显示器
磁盘

存储器的体系结构

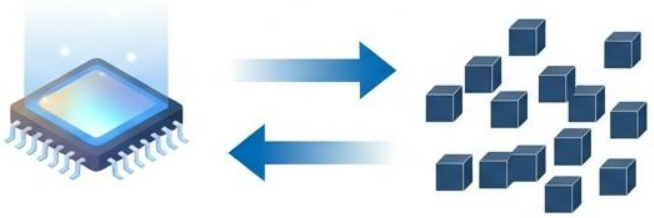
存储与计算的瓶颈问题



延迟 vs 带宽

系统存储性能对比

内存（SRAM/DRAM）：延迟是生命线



CPU直接工作台

随机、频繁、小块

每次等待时间直接叠加
降低延迟是关键

低延迟

硬盘（HDD）：带宽更能体现价值



仓库（长期存储）

顺序、偶发、大块

付出固定延迟后，衡量
搬运整箱数据的速度

高带宽

1. 向DRAM请求：CPU等100纳秒，然后以50GB/s的速率收到数据。等待是主要开销。
2. 向硬盘请求：CPU等10毫秒（10,000,000纳秒），然后以200MB/s的速率收到数据。
3. 延迟决定了系统的最低响应时间，而带宽决定了数据管道的最大流量。

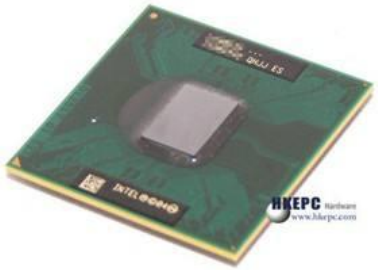
存储器






Intel SSD 910 Series
800GB


numonyx
Omneo™ P5Q PCM
numonyx
Omneo™ P8P PCM


HKEPC


Kingston
SMS200S3/120G
120GB4000008
S5FAM008 - 1117
Made in TAIWAN
DC+3.3V 0.35A
ATA

存储器属性

- 靠CPU侧：SRAM + DRAM

TABLE I: Characteristics of Different Types of Memory

Category	Read Latency (*ns*)	Write Latency (*ns*)	Endurance (*# of writes per bit*)
SRAM	2-3	2-3	∞
DRAM	15	15	10^{18}
STT-RAM	5-30	10-100	10^{15}
PCM	50-70	150-220	10^{8}-10^{12}
Flash	25,000	200,000-500,000	10^{5}
HDD	3,000,000	3,000,000	∞

内存地址问题？


```
(gdb) l
1       #include <stdio.h>
2
3       int j = 1;
4
5       void foo() {
6           int i = 0;
7           printf("%d\n", i);
8           printf("%d\n", j);
9       }
10
11      int main(int argc, char **argv) {
12          foo();
13          return 0;
14      }
```

```
Breakpoint 1, foo () at example.c:7
7           printf("%d\n", i);
(gdb) p i
$1 = 0
(gdb) p &i
$2 = (int *) 0x7fffffffe49c
(gdb) c
Continuing.
0

Breakpoint 2, foo () at example.c:8
8           printf("%d\n", j);
(gdb) p j
$3 = 1
(gdb) p &j
$4 = (int *) 0x555555558010 <j>
```

- **这两个地址的值对应多大的内存地址空间？**
- **内存不够怎么办？内存到底是如何管理的？**

本章主要内容

- 1 背景
  - （1）程序的装入、运行和地址映射；
  - （2）逻辑地址空间和物理地址空间
  - （3）进程内外存交换
- 2 连续内存分配
  - （1）固定等长分区、固定变长分区、可变分区、分区分配算法；
  - （2）碎片问题与内存紧缩
- 3 分段内存管理
- 4 分页内存管理
- 5 段页结合内存管理

背景

- 内存是做什么用的？
- 程序是如何装入内存的？
- 数据在内存中是如何组织的？
- 程序在运行中地址是如何管理的？
- 内存不够用怎么办？

再回到那个恒久的话题

- 执行程序是计算机的基本任务

```
int main(int argc, char* argv[])
{
      int  i, to, sum = 0;
      to = atoi(argv[1]);
      for(i=1; i<=to; i++)
      {
            sum = sum + i;
      }
      printf("%d", sum);
}
```



```
C:\>sum 3127
4890628
C:\>sum 6656
22154496
C:\>sum 12345
76205685
C:\>sum 32178
517727931
C:\>_
```

- 静态链接？动态链接？

让程序执行起来就成了最重要的事!

第一步: 编译——从C到汇编

```
int main(int argc, char*
argv[])
{
      int  i , to, sum = 0;
      to = atoi(argv[1]);
      for(i=1; i<=to; i++)
      {
            sum = sum + i;
       }
       printf("%d", sum);
}
```

源代码

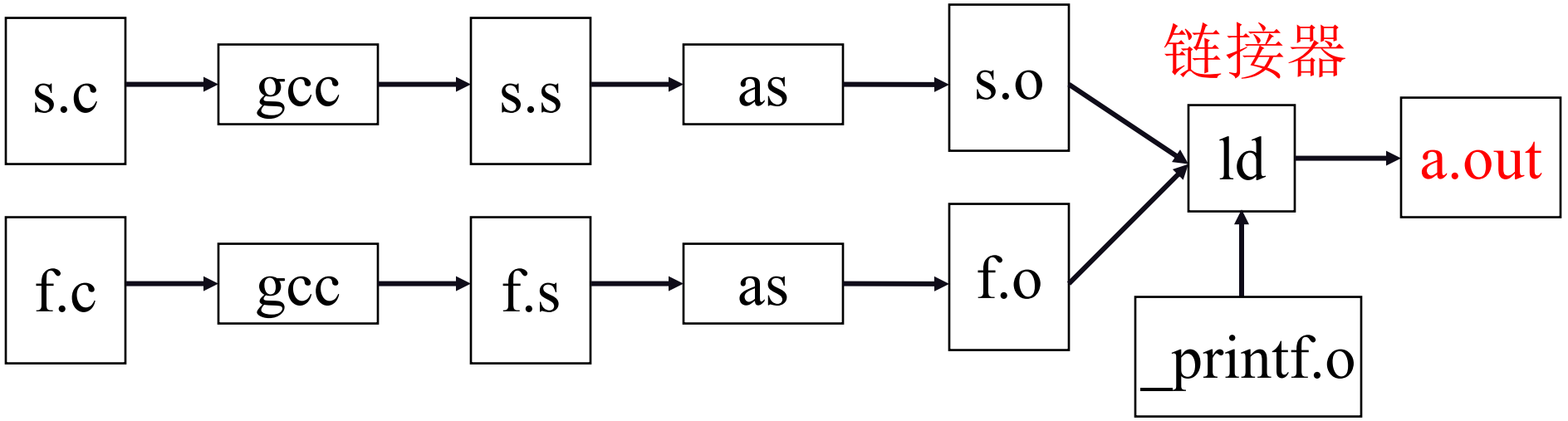
```
.text
_entry: //入口地址
      mov ax,  [8+sp]
      mov [_environ], ax
      call _main
      push ax  //main返回值
      call _exit
_main:
      mov [_sum],  0
      sub sp, 4
      mov [sp+4], [_environ+4]
      call _atoi
      mov [_to],ax//atoi返回ax
      mov [_i], 1
```

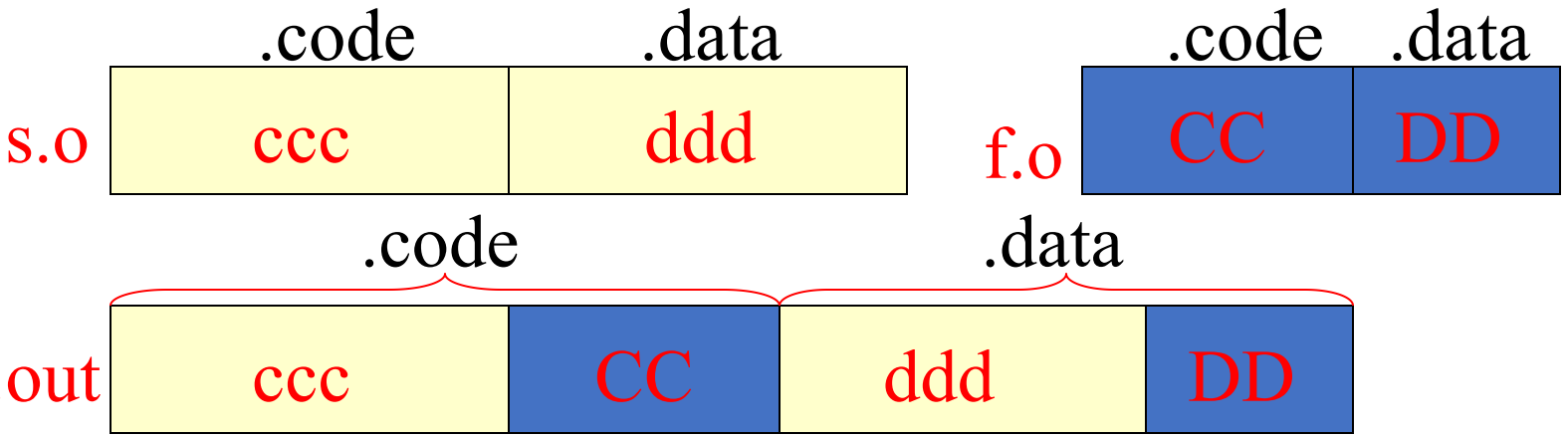
```
1:
      j> 2f, [_i], [_to]
      mov ax, [_sum]
      add ax, [_i]
      mov [_sum], ax
      add [_i], 1
      jmp   1b
2:    sub sp, 8
      mov [sp+4], [_sum]
      call _printf
      ret
.data:
      _environ:   .long 0
      _i:   .int  0
      _to:   .int  0
      _sum:   .int  0
```

汇编代码

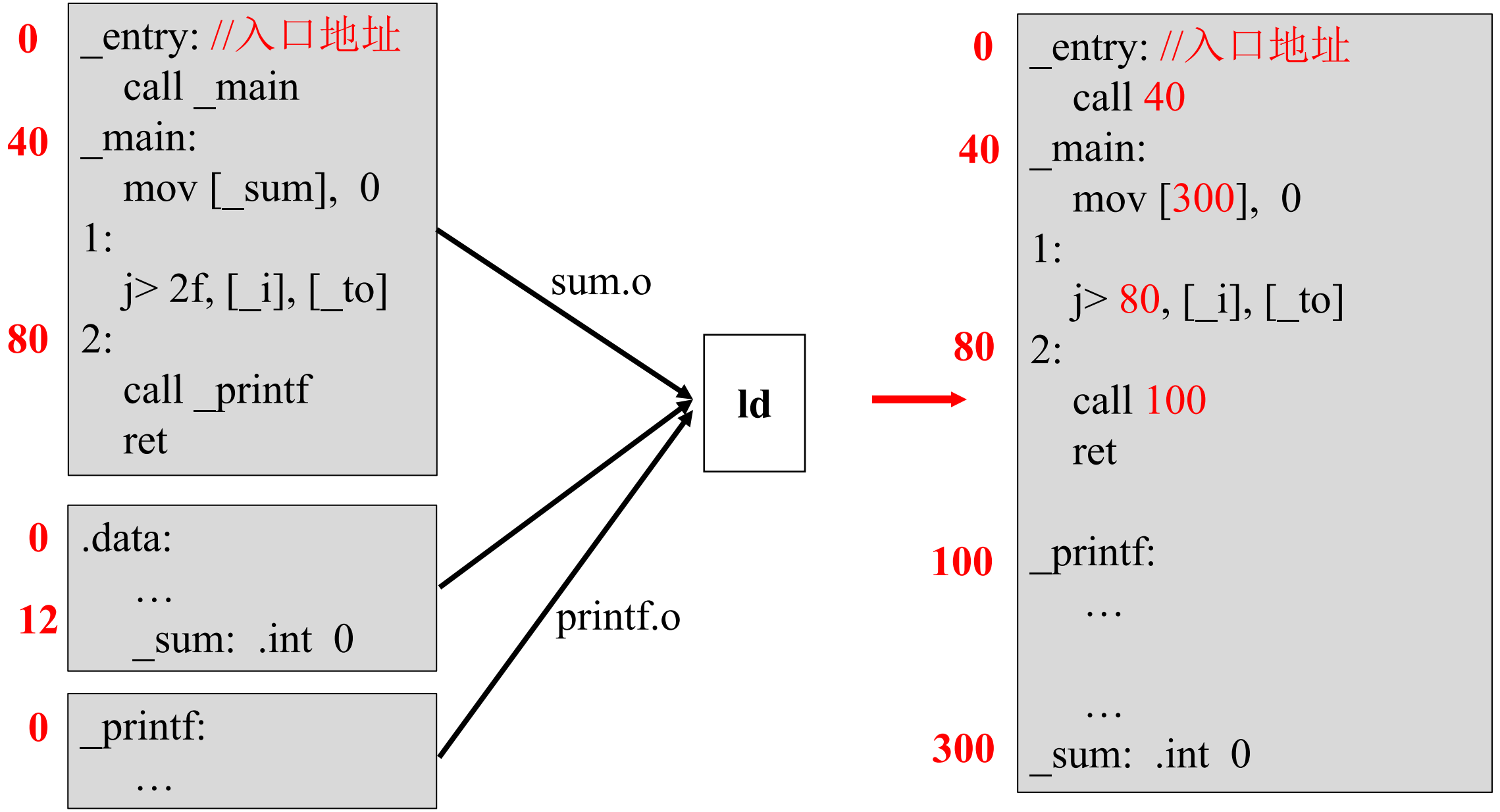
许多东西有待明确…

- 第二步: 汇编与链接——从汇编到可执行程序





前面的程序经过链接以后…

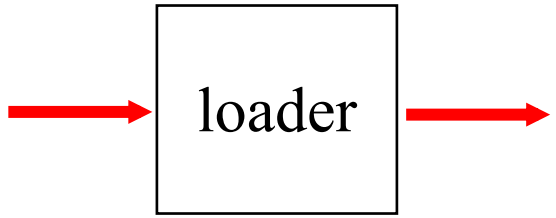


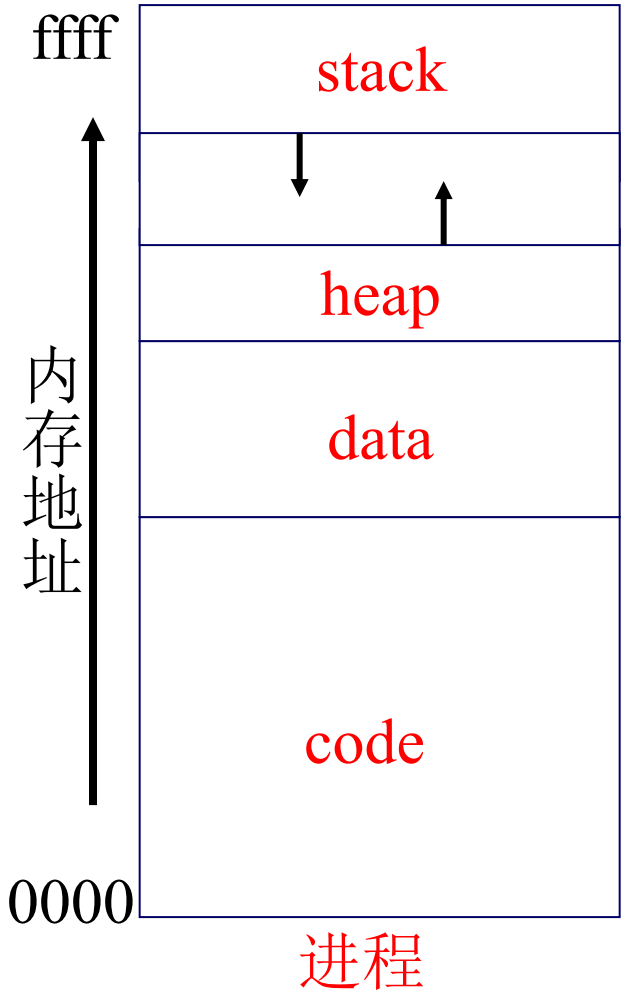
现在还是程序，不是进程


```
0    _entry: //入口地址
        call 40
40   _main:
        mov [300],  0
     1:
        j> 80, [_i], [_to]
80   2:
        call 100
        ret

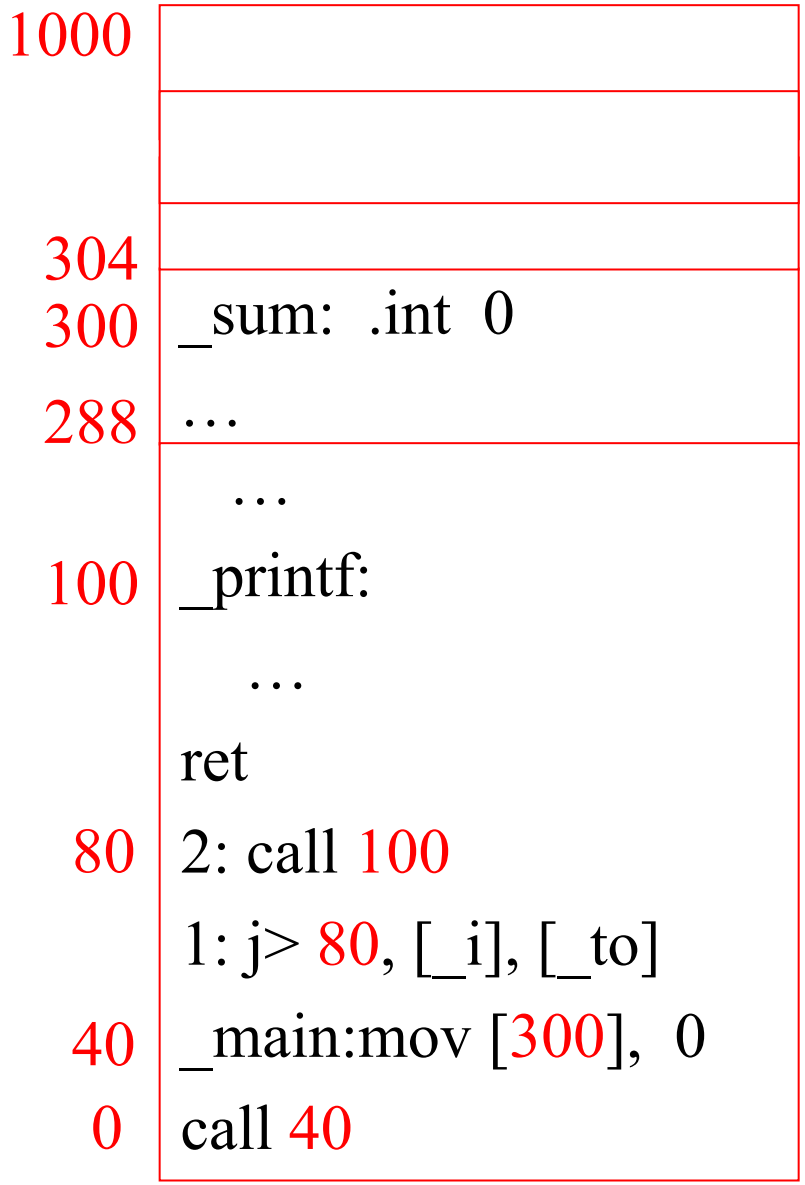
100  _printf:
        …

        …
300  _sum:  .int  0
```





程序可以执行了吗?



- 程序怎么能正确开始?
- 将代码段放在内存中从0开始的地方
- 将数据段放在内存中从288开始的地方
- 设置PC=0
- 如果内存中从0开始的一段内存有专门的用途怎么办？例如：存放中断处理程序

需要重定位!

1000是由硬件和操作系统决定的!

- 假设内存从地址1000以后是可以使用的

```
1300  _sum:  .int  0
1288  …
      …
1100  _printf:
      …
      ret
1080  2:call 100
      1: j> 80, [_i], [_to]
1040  _main:mov [300], 0
1000  call 40
```

重定位

```
1300  _sum:  .int  0
1288  …
      …
1100  _printf:
      …
      ret
1080  2:call 1100
      1: j> 1080, [_i], [_to]
1040  _main:mov [1300], 0
1000  call 1040
```

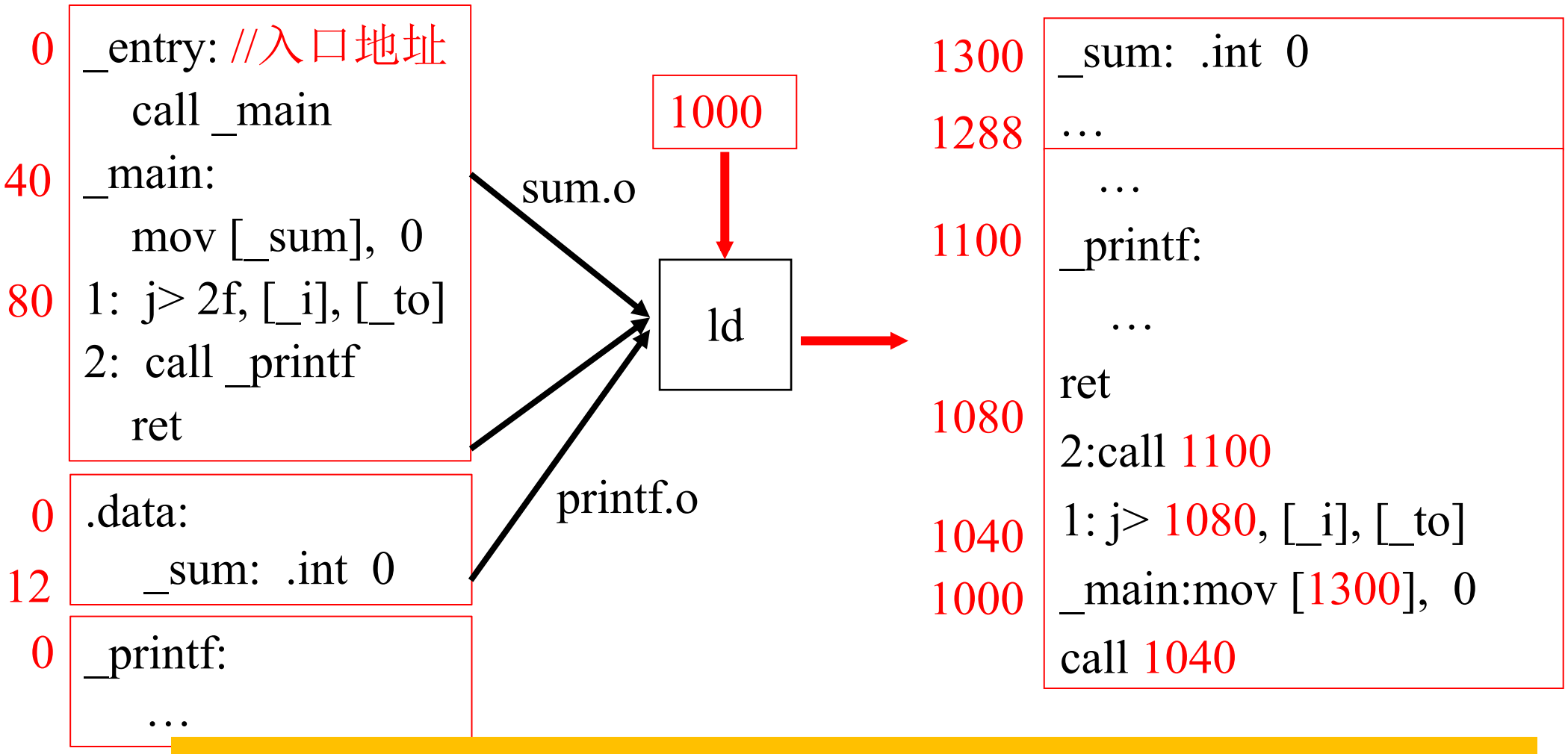
重定位: 为执行程序而对其中出现的地址所做的修改

重定位可以执行的时机

濱工業大學（深圳）
INSTITUTE OF TECHNOLOGY, SHENZHEN

这样的代码叫绝对代码!

- 第一种时机: 在编译链接时



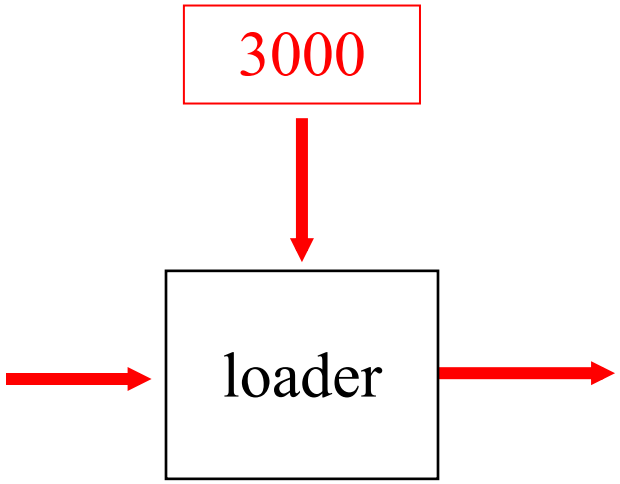
绝对代码：这样的代码只能放在事先确定的位置上

并发 =>多个程序同时在内存中

- 分别用1000，2000，… 吗？第二种时机: 载入时

可重定位代码!

```
_sum:  .int  0
…
   …
_printf:
    …
ret
2:call 100
1: j> 80, [_i], [_to]
_main:mov [300],  0
call 40
```



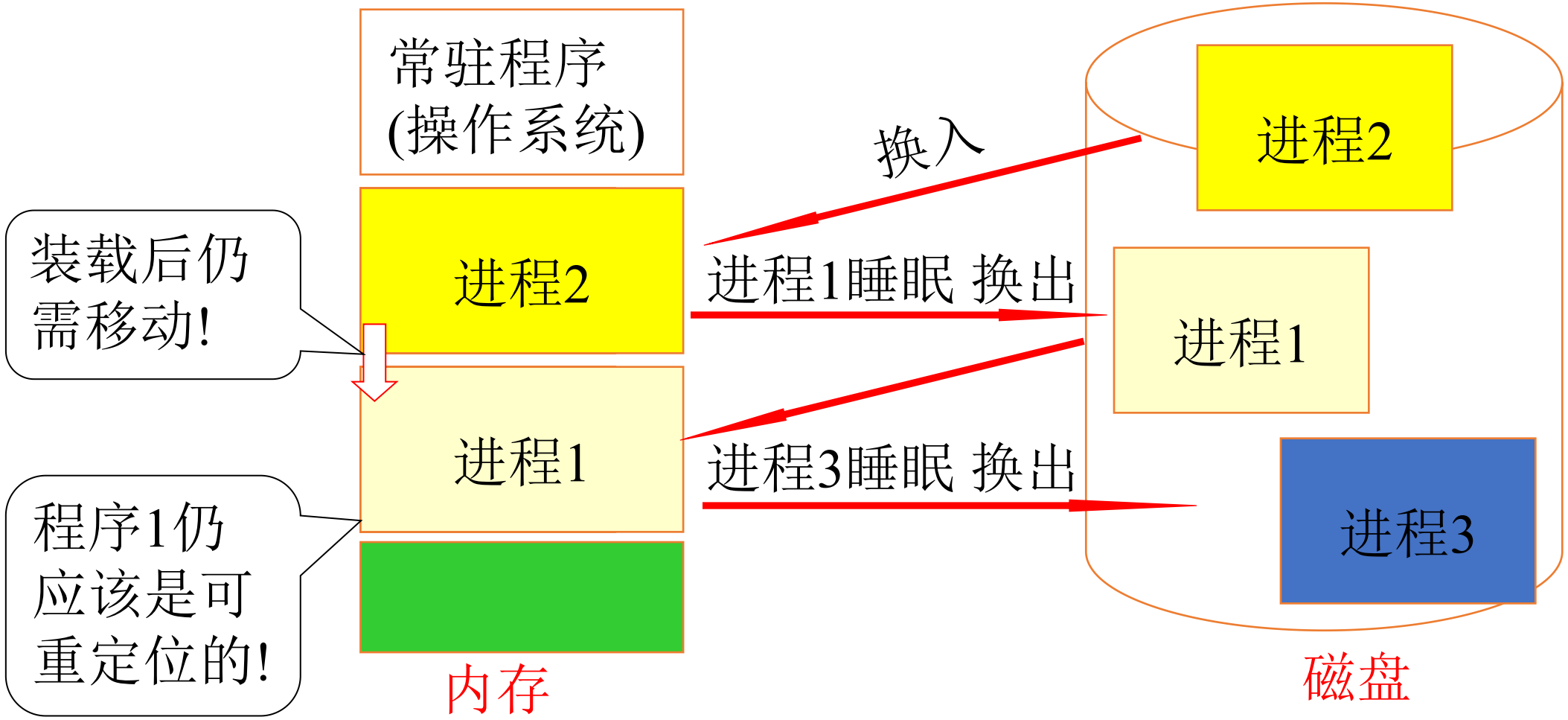

```
3300  _sum:  .int  0
3288  …
         …
3100  _printf:
          …
      ret
3080  2:call 3100
      1: j> 3080, [_i], [_to]
3040  _main:mov [3300],  0
3000  call 3040
```

装载时的重定位仍然存在缺点…一旦载入不能移动

移动也是很有必要的!

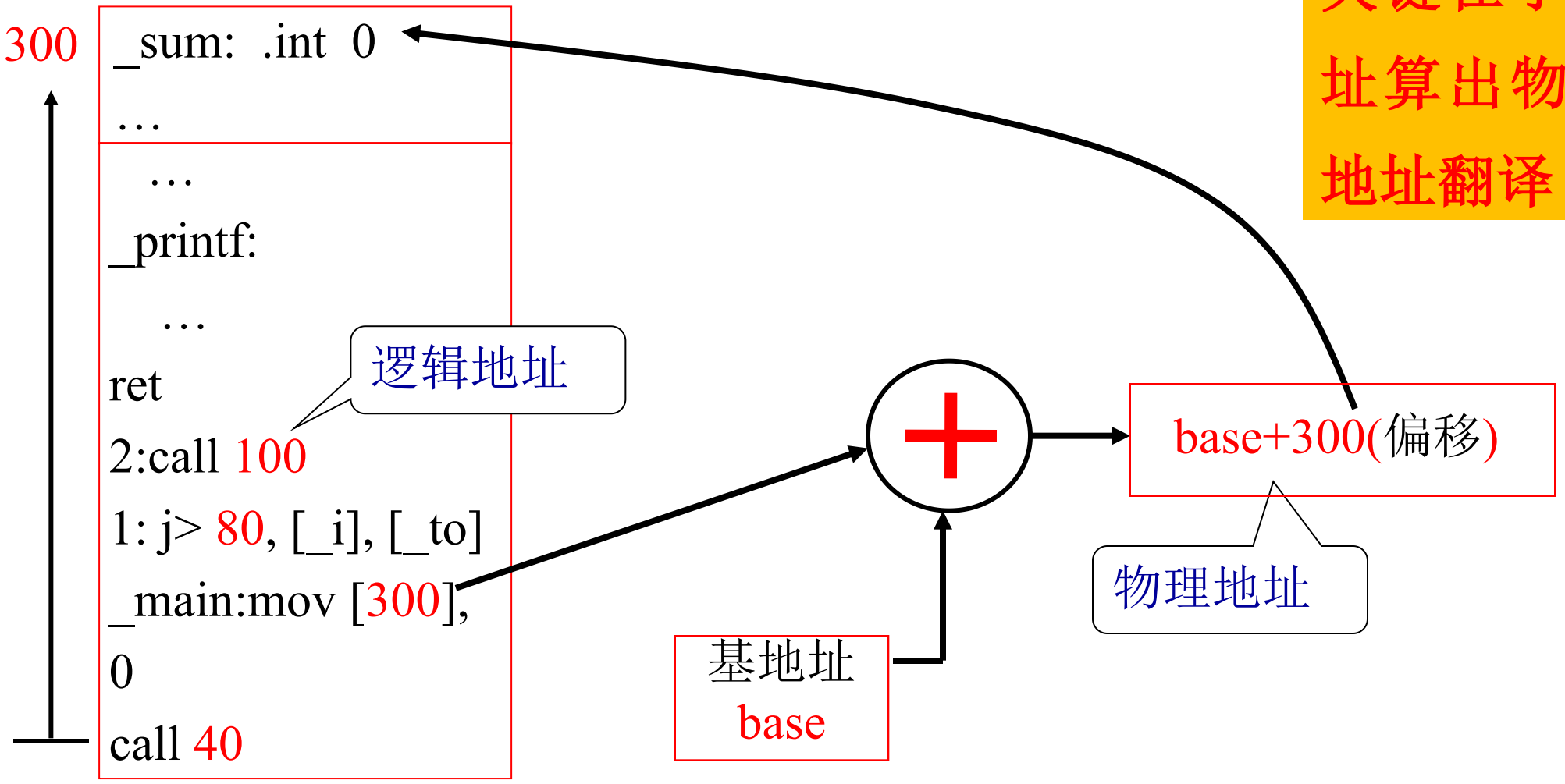
- 一个重要概念: 交换(swap)
- 能让更多的进程并发，提高内存的利用率

如何来实现呢？



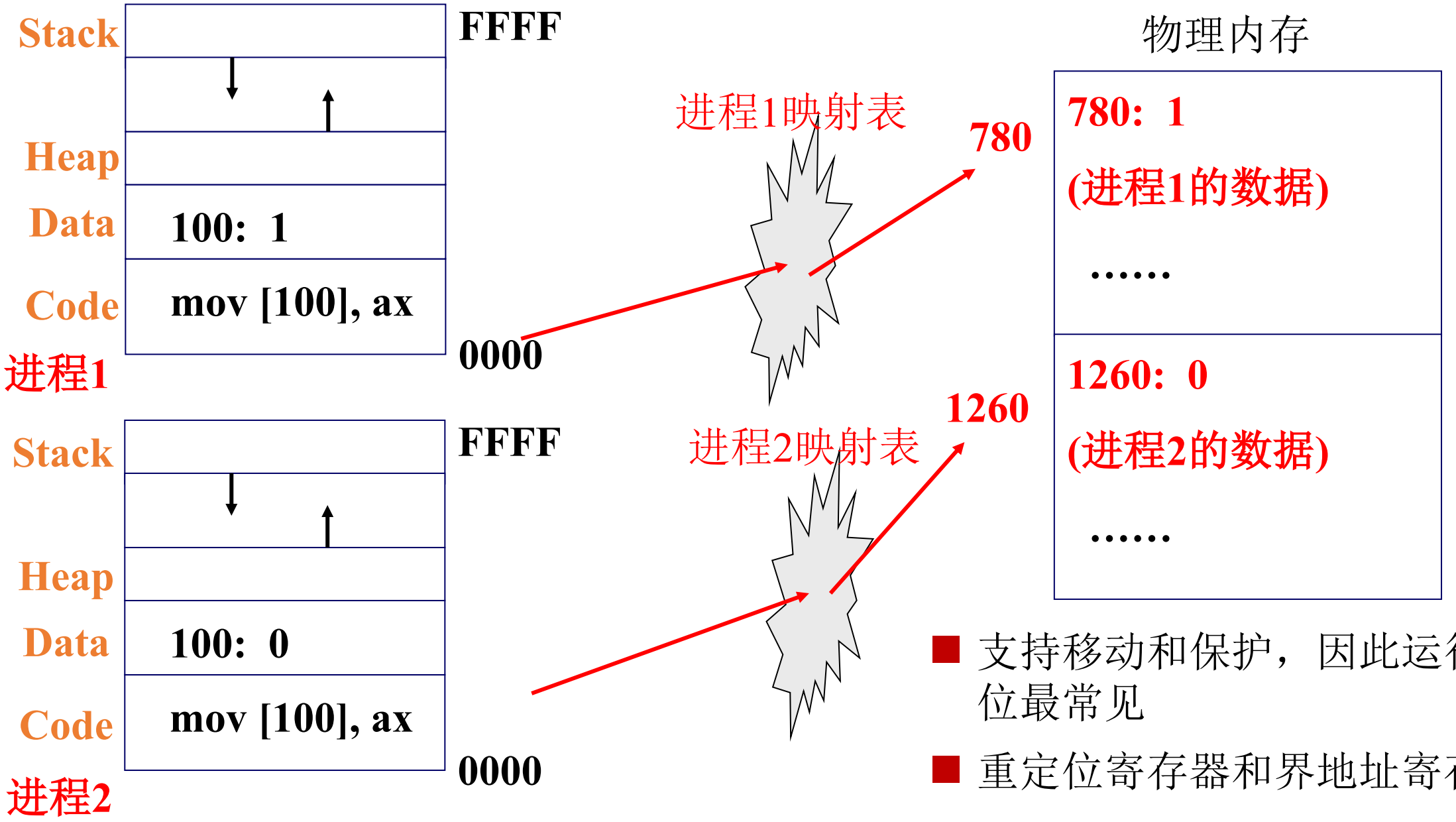
重定位最合适的时机--运行时重定位

- 内存中的代码总是可重定位的!



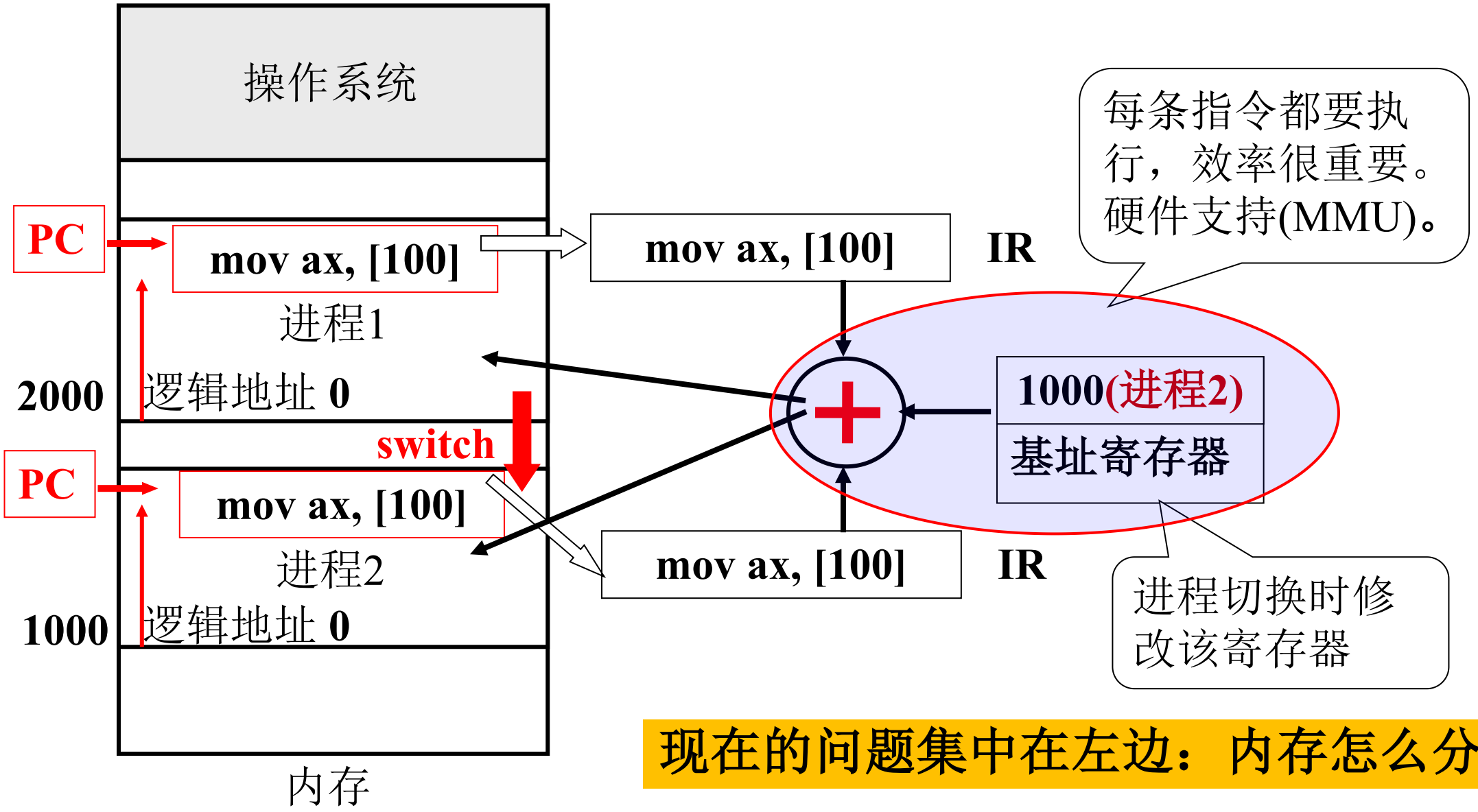
关键在于从逻辑地址算出物理地址：地址翻译

运行时重定位还有一个好处：进程保护



- 支持移动和保护，因此运行时重定位最常见
- 重定位寄存器和界地址寄存器

整理一下思路...



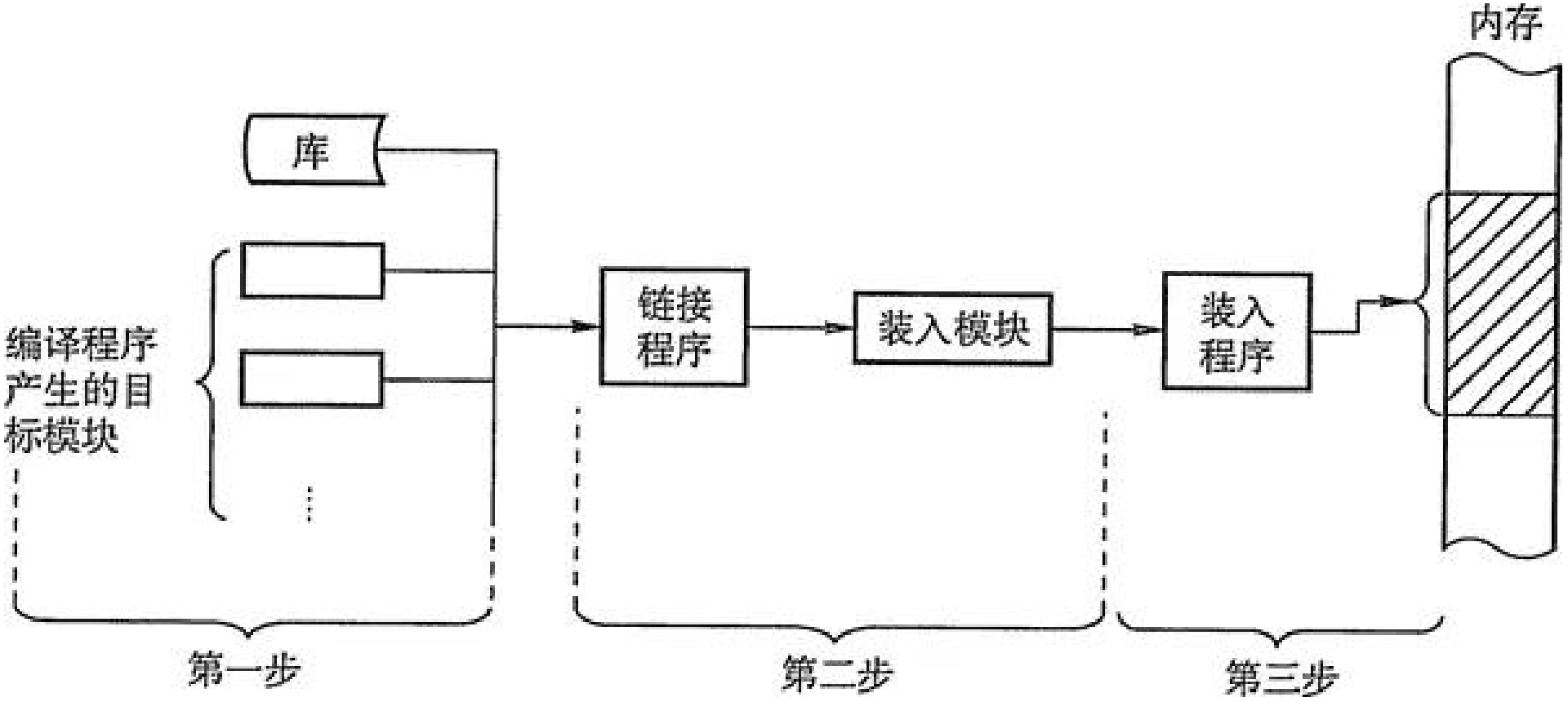
现在的问题集中在左边：内存怎么分配?

简单总结一下

- 操作系统将可执行程序加载到内存，合理分配内存。
- 可执行程序经过编译产生了逻辑地址，为了提高进程的并发和内存利用率，逻辑地址和运行物理地址间需要地址转换。
- 嵌入式操作系统多数内存管理非常简单，甚至没有内存管理。
- PC和服务器、分布式等现代操作系统基本都支持虚拟内存管理。

简单总结一下（续）

- 创建进程首先要将程序和数据装入内存。将用户源程序变为可在内存中执行的程序，通常需要编译、链接、装入三个步骤



第1组3个重要概念

- 虚拟地址：用户编程时将代码（或数据）分成若干个段，每条代码或每个数据的地址由段名称 + 段内相对地址构成，这样的程序地址称为虚拟地址
- 逻辑地址：虚拟地址中，段内相对地址部分称为逻辑地址
- 物理地址：实际物理内存中所看到的存储地址称为物理地址
- 比较：虚拟地址由用户编写程序时定义的全局地址；逻辑地址是用户定义的局部地址，是虚拟地址的组成部分；物理地址就是实际存在的以Byte为单位的存储单元编号

第2组3个重要概念

- 逻辑地址空间：

  逻辑地址空间是指一个源程序在编译或者链接装配后指令和数据所用的所有相对地址的空间。它是作业进入内存，其程序、数据在内存中定位的参数。逻辑地址的集合称为逻辑地址空间

- 线性地址空间：

  CPU地址总线可以访问的所有地址集合称为线性地址空间

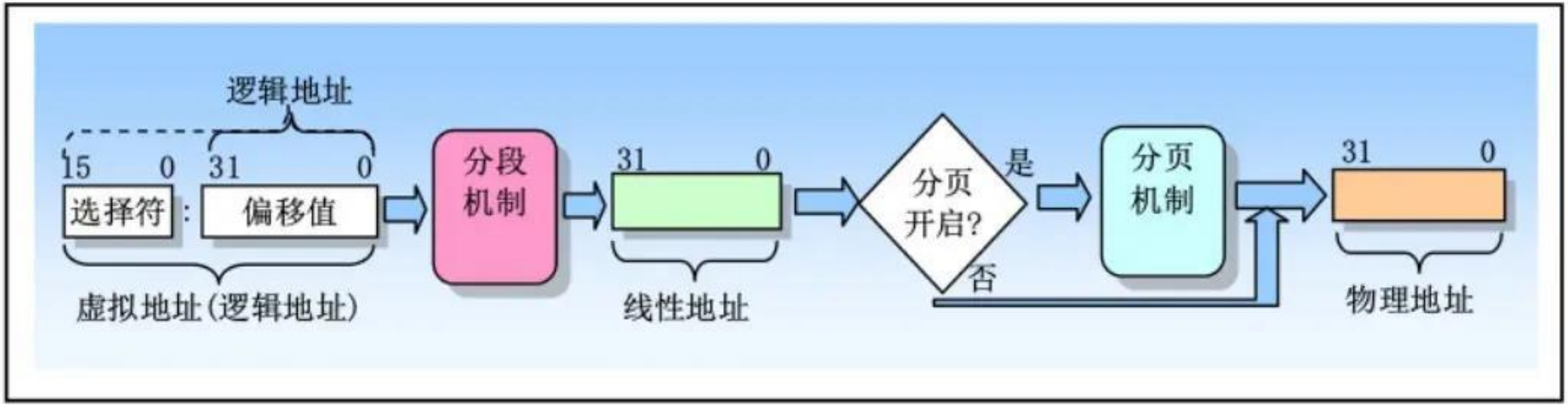
  一般也指虚拟地址空间

- 物理地址空间：

  实际存在的可访问的物理内存地址集合称为物理地址空间

逻辑地址空间-线性地址空间-物理地址空间的关系

经典教材约定的基于x86架构的OS采用如下的地址变换运算：逻辑地址->物理地址



注意：在实际Linux系统中，取消了分段机制（分段机制是x86向下兼容的历史包袱），虚拟地址、逻辑地址、线性地址经常不加区分，可统称为虚拟地址。

第3组3个重要概念

- MMU（Memery Management Unit内存管理单元）：

  实现将用户程序的虚拟地址（逻辑地址）→ 物理地址映射的CPU中的硬件电路（配合操作系统工作）

- 基地址：

  在进行地址映射时，经常以段或页为单位并以其最小地址（即起始地址）为基值来进行计算

- 偏移量：

  在以段或页为单位进行地址映射时，相对于基地址的地址值

虚拟地址与物理地址

- 嵌入式操作系统多数内存管理非常简单，甚至没有内存管理。虚拟地址=物理地址
- PC和服务器、分布式等现代操作系统基本都支持虚拟内存管理。虚拟地址!=物理地址

- 前者链接时重定位
- 后者运行时重定位

内存分配与管理方案

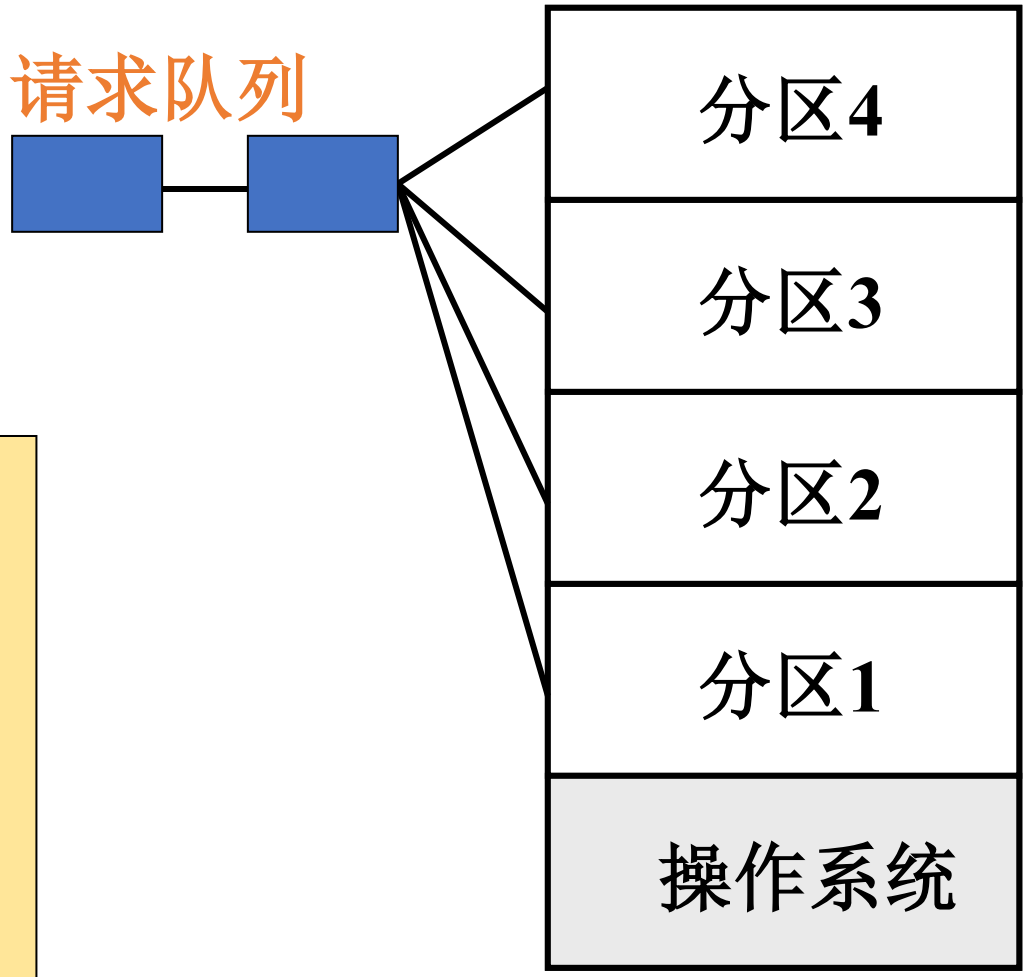
- 连续内存分配与管理
- 分段分配与管理
- 分页分配与管理
- 段页结合管理
- 请求式分页管理(虚拟内存)

从最简单的分配方案开始!

(1) 固定分区--等长分区

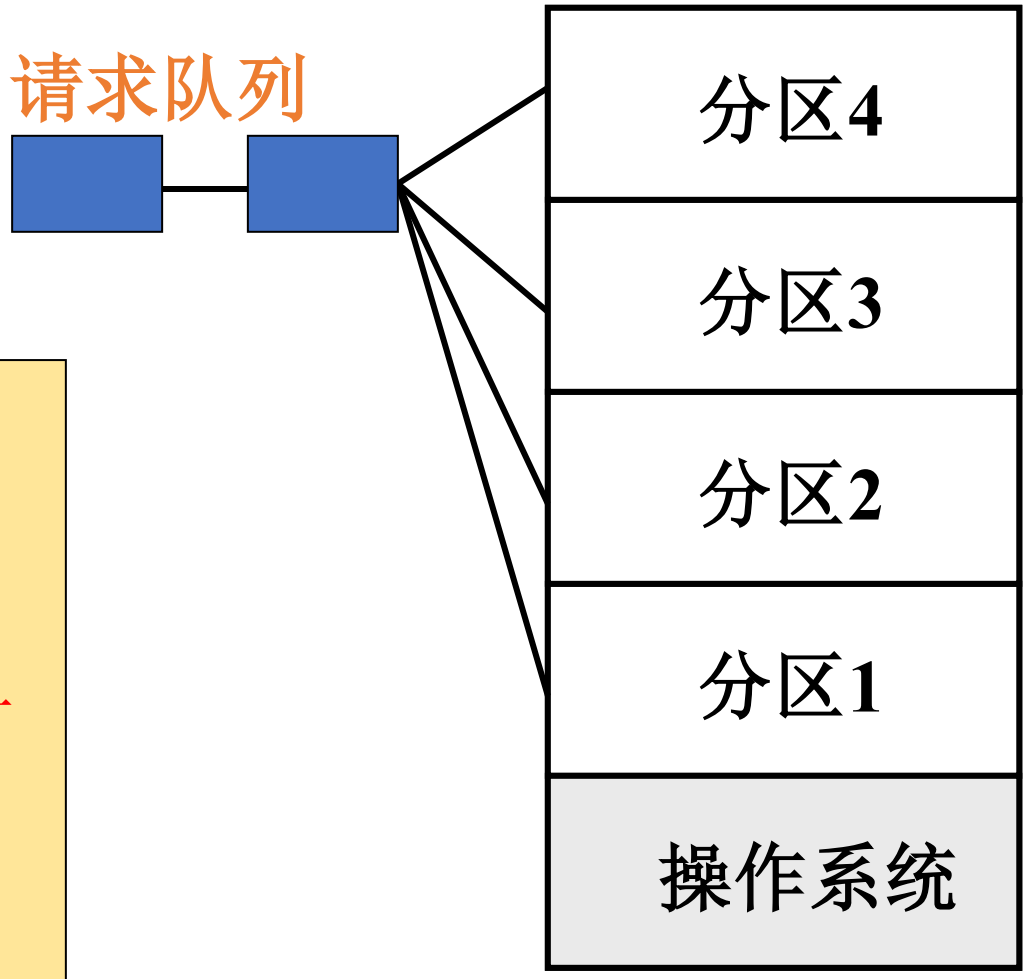
- 给你一个面包，n个孩子来吃，怎么办?
- 等分…
- 操作系统初始化时将内存等分成k个分区

```
bool AvailSec[k]; int SecSize;
内存请求算法   //进程创建时
  1. if(reqSize>SecSize) exit
  2. 找出AvailSec[i]为真的i
  3. 如果有，返回分区i的基址
  4. 否则，将current加入请求队列等待
```



(2)固定分区--变长分区

- 孩子有大有小，进程也有大有小...
- 初始化时将内存分成k个大小不同的分区



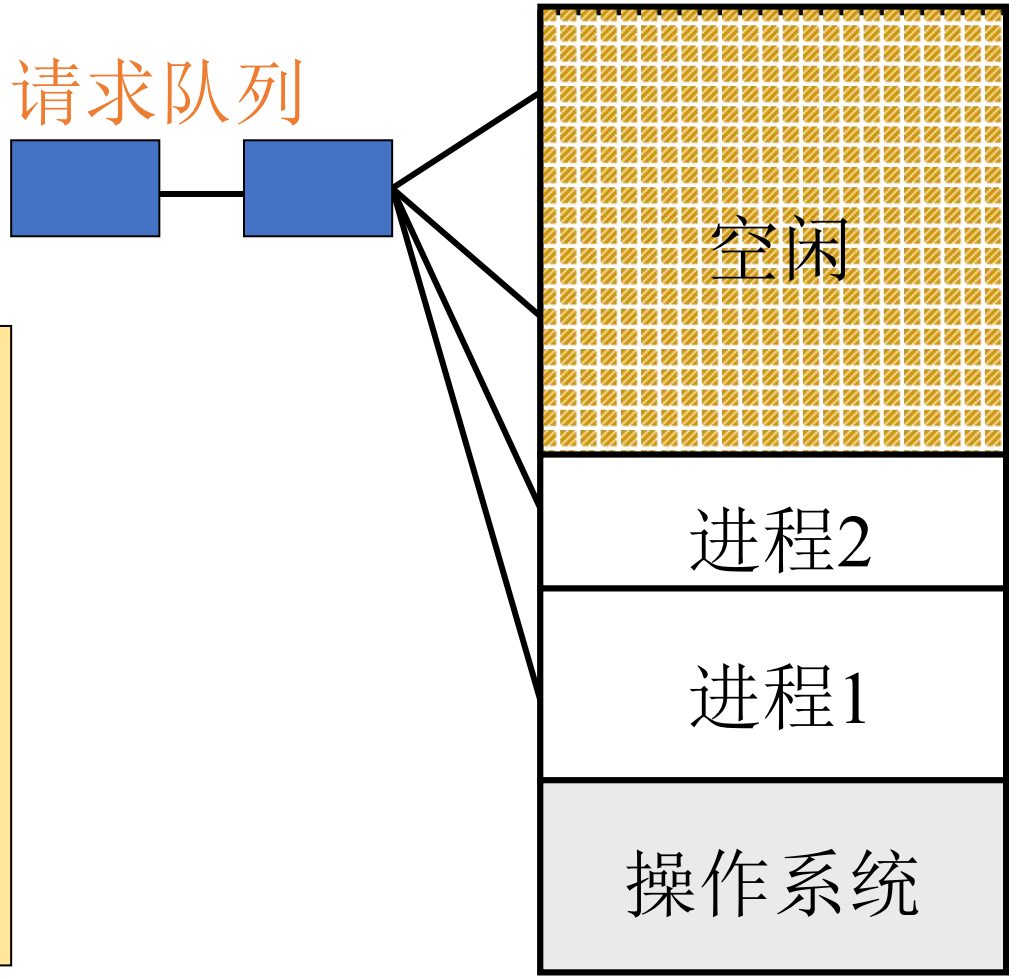

```
struct Section AvailSec[k];
内存请求算法   //进程创建时
  1. if(reqSize>MaxSize) exit
  2. 找出AvailSec[i].Size>reqSize且
AvailSec[i].Size最小的空闲分区i
  3. 如果有，返回分区i的基址
  4. 否则，将current加入请求队列等待
```

(3)可变分区

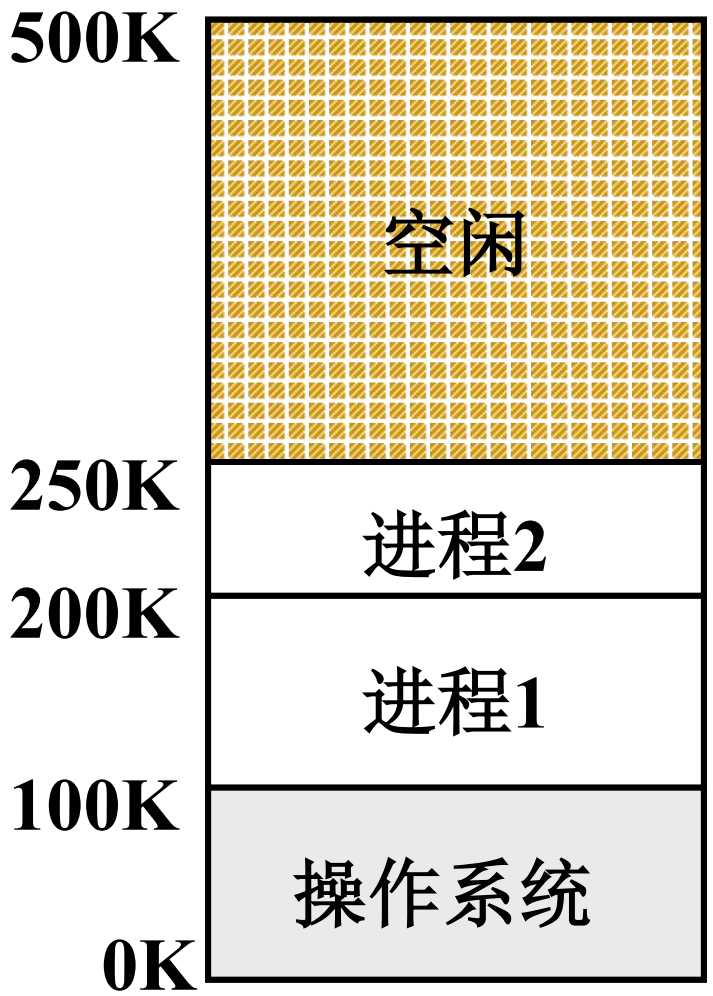
- 合理的方法应该是根据孩子饥饿程度来分割
- 根据reqSize进行动态分割

内存请求算法　//进程创建时

1. if(reqSize>内存大小)exit
2. if(reqSize>空闲空间总尺寸)
   将current加入请求队列等待（**阻塞**）
3. 从空闲分区划出一个reqSize,并返回其基地址　//哪个空闲分区？
4. 修改分区数据结构



可变分区的数据结构



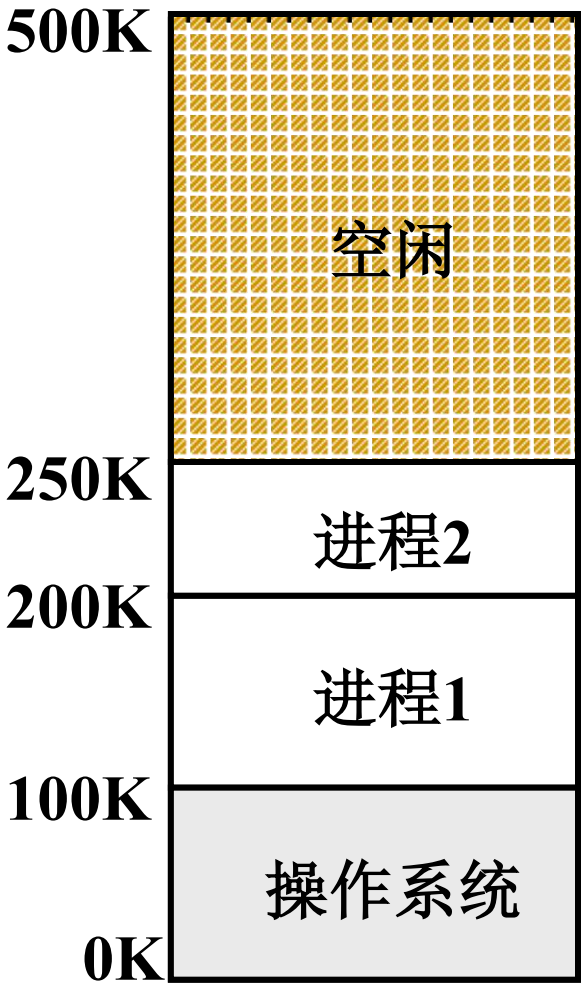
空闲分区表

始址	长度
250K	250K

已分配分区表

始址	长度	标志
0K	100K	OS
100K	100K	P1
200K	50K	P2

可变分区数据结构的变化(1)



⑩ 内存请求: reqSize = 100K

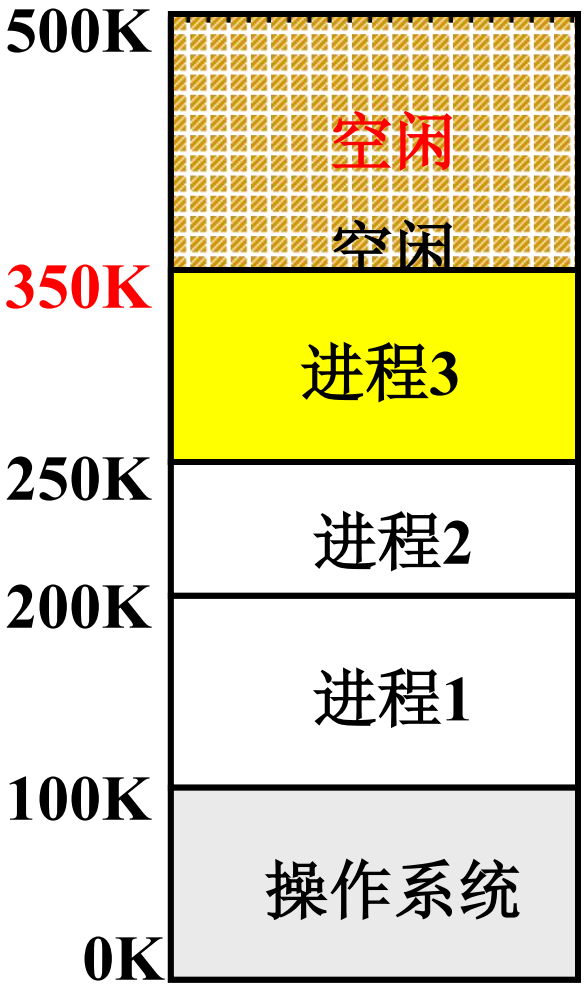
空闲分区表

始址	长度
250K	250K

已分配分区表

始址	长度	标志
0K	100K	OS
100K	100K	P1
200K	50K	P2

可变分区数据结构的变化(1)



⑩ 内存请求: reqSize = 100K

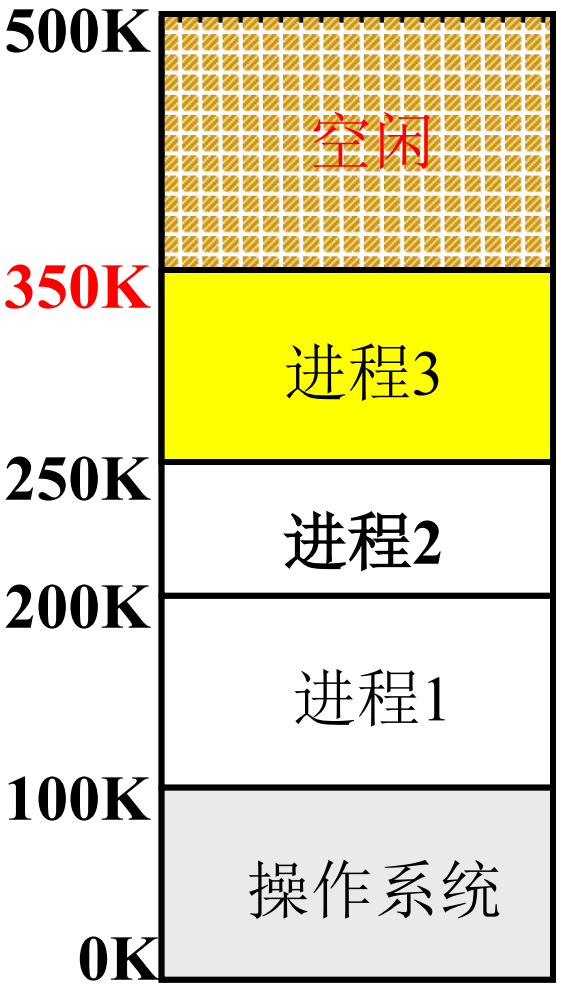
空闲分区表

始址	长度
350K	150K

已分配分区表

始址	长度	标志
0K	100K	OS
100K	100K	P1
200K	50K	P2
250K	100K	P3

可变分区数据结构的变化(2)



⑩ 进程2执行完毕，释放内存

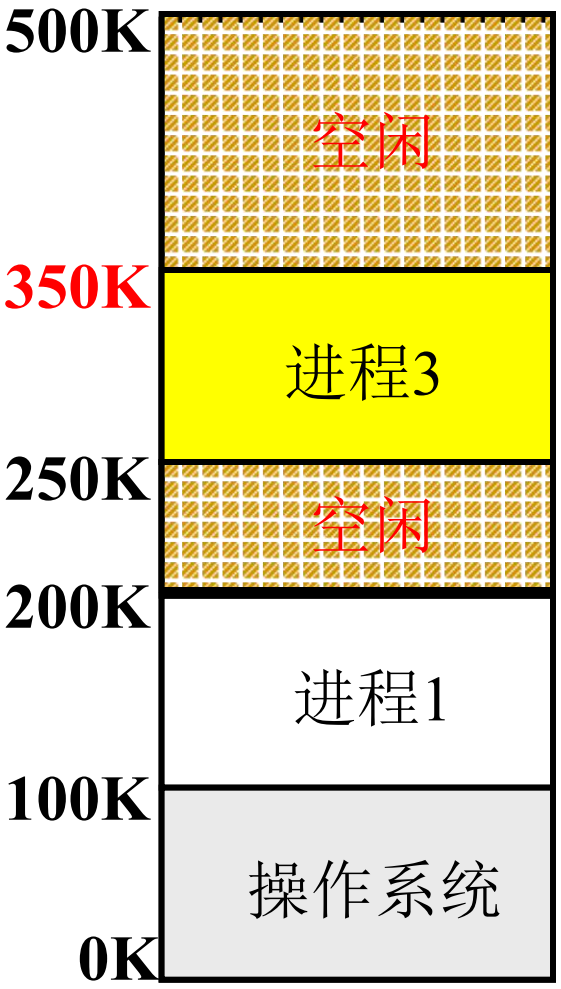
空闲分区表

始址	长度
350K	150K

已分配分区表

始址	长度	标志
0K	100K	OS
100K	100K	P1
200K	50K	P2
250K	100K	P3

可变分区数据结构的变化(2)



⑩ 进程2执行完毕，释放内存

空闲分区表

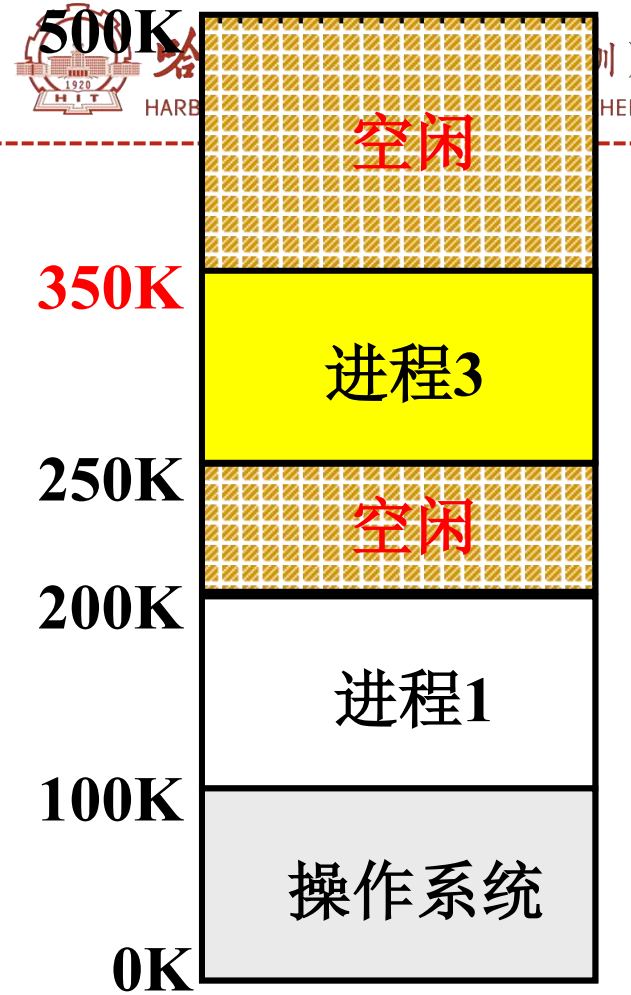
始址	长度
350K	150K
200K	50K

已分配分区表

始址	长度	标志
0K	100K	OS
100K	100K	P1
250K	100K	P3

(4)分区分配算法--找到合适的空闲分区

- 发起请求reqSize=40K怎么办?
- 首次适配(first-fit):首次找到的满足要求的空闲分区(350,150)。特点：快速！
- 最佳适配(best-fit): 查找最小的满足要求的空闲分区(200,50)。特点:(1)搜索整个空闲分区表，慢！(2)会产生许多小的空闲分区
- 最差适配(worst-fit):查找最大的满足要求的空闲分区(350,150)。特点:(1)搜索整个空闲分区表，慢！(2)新产生的空闲分区大一些



空闲分区表

始址	长度
350K	150K
200K	50K

(5)碎片问题

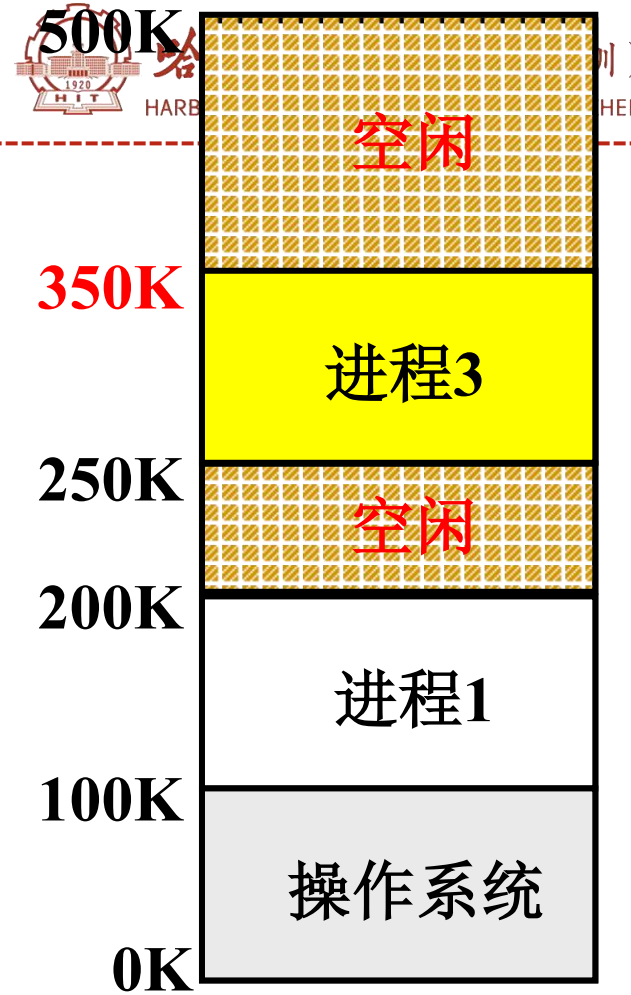
- 碎片：内存中剩余的无法使用的存储空间
- 外部碎片：随着进程不断的装入和移出，对分区不断地分割，使得内存中产生许多特别小的分区，它们并不连续可用
- 内部碎片：对固定分区来说，只要分区被分配给某进程使用，其中并未占用的空间不能分给其他进程
- 碎片的产生和分配“制度”有关！不易避免！
- 碎片问题可以消除（内存紧缩），但时间成本太高！
- 磁盘也存在碎片问题！

(6)外部碎片处理--内存紧缩

- 发起请求reqSize=160K怎么办?
- 总空闲空间>160，但没有一个空闲分区>160，怎么办?
- 内存紧缩: 将空闲分区合并在一起，需要移动进程3(复制内容)
- 内存紧缩需要花费大量时间，如果一台2GB 内存的计算机可以每20ns复制4个字节，它紧缩1GB内存大约花费多久？？

该值表明连续分配技术不合适!

磁盘碎片是什么？

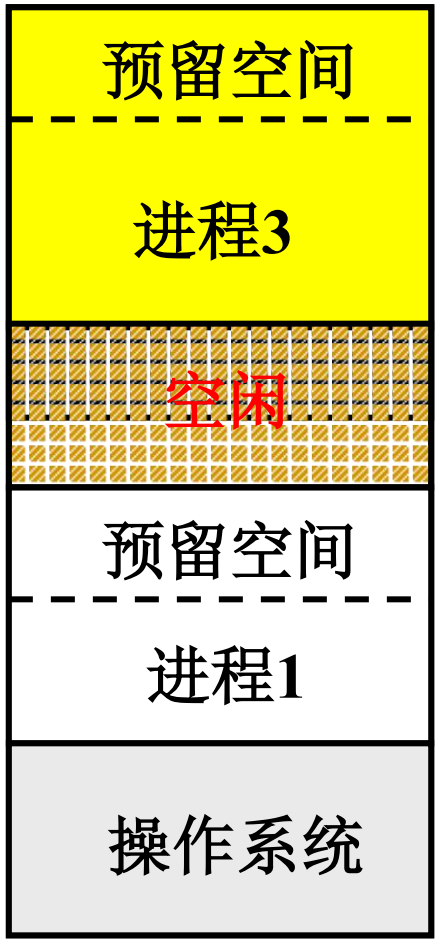


空闲分区表

始址	长度
350K	150K
200K	50K

分区方案中还有一个问题没有搞清楚

- reqSize值怎么确定?
- reqSize=code+data+stack+heap
- code和data不难处理，stack和heap难处理：动态增长，预先不知道
- 怎么办? 预留空间
- 预留空间用完了怎么办? 找一个更大的空闲空间，移动该进程
- 实际上移动code和data纯粹是浪费
- 怎么办? 各个段区别对待，分别分配

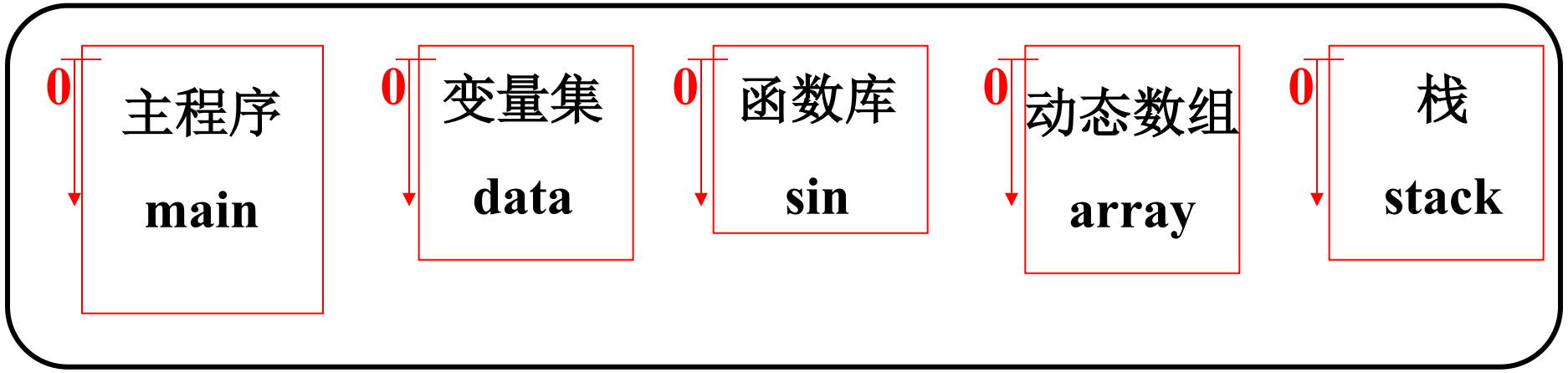


还有利于建立合适的保护策略!

分段(Segmentation)!

程序员眼中的程序（从汇编—高级语言）

- 由若干部分(段)组成，每个段有各自的特点、用途!

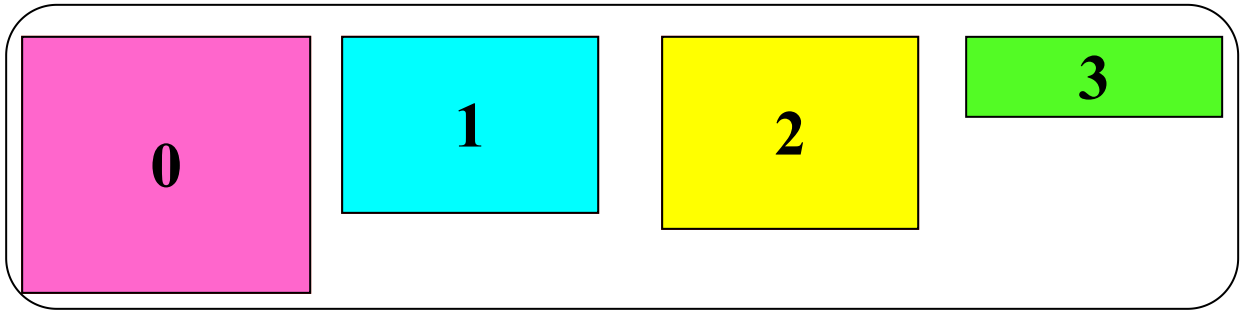


程序员眼中的一个程序

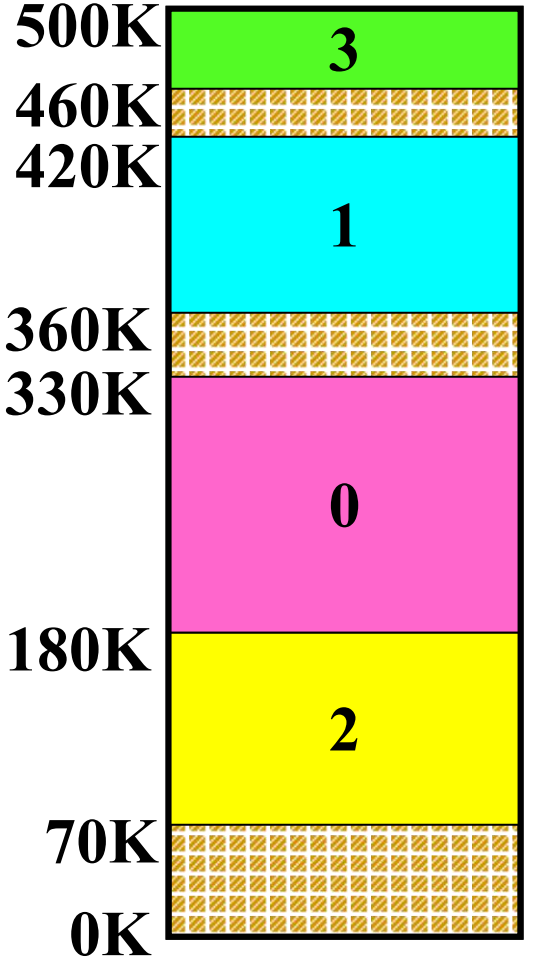
- 程序员怎么定位具体指令(数据): <段号, 段内偏移>
- 分段符合用户观点: 用户可独立考虑每个段(分治)

将段放入内存

- 分段制造了二维空间，而内存是一维的



各个段可分散



- 一个进程需要记录多个基址…

进程段表

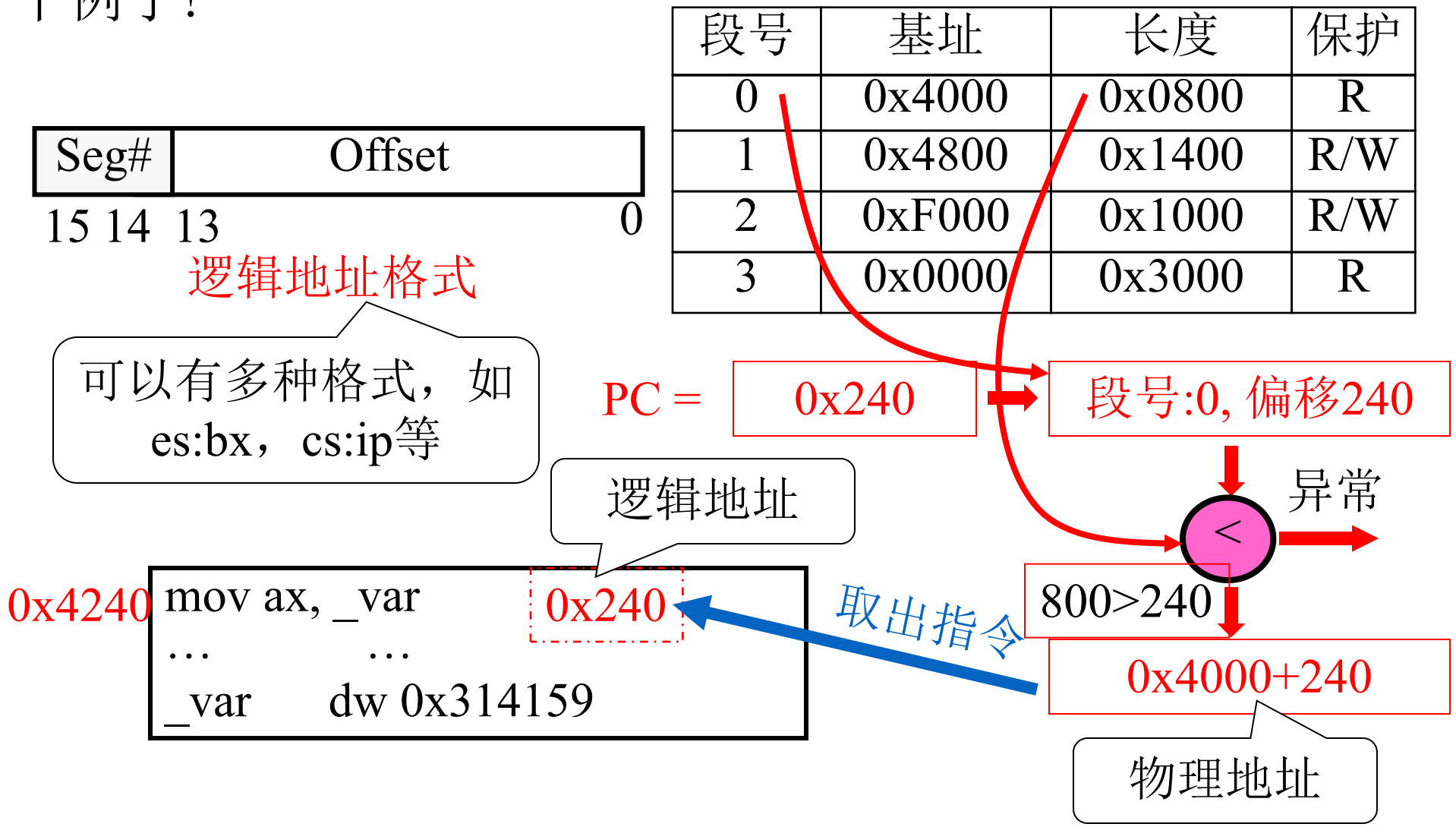
段号	基址	长度	保护
0	180K	150K	R
1	360K	60K	R/W
2	70K	110K	R/W
3	460K	40K	R

引入段表

- 仍有内存分区表，内存分配算法等等

分段的地址翻译

- 来看一个例子!



段地址映射计算方法

- 段号S和段内偏移O的计算公式
  - S＝INT [A/L]　　INT：整除函数
  - O＝[A] % L　　%：取余
  - A：逻辑地址，L：逻辑段大小



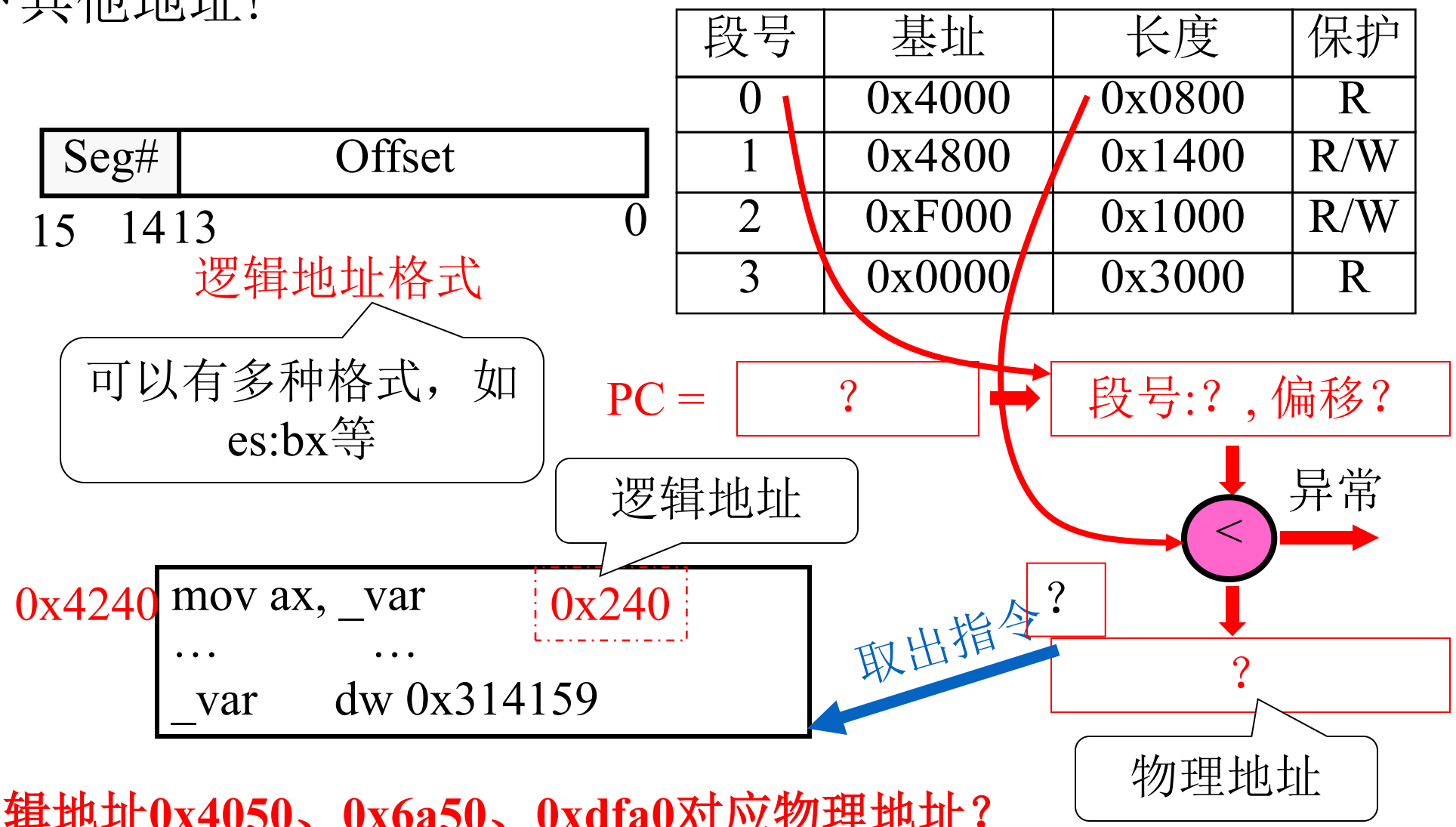
逻辑地址格式

- 例如：假定段大小为2^{14}B，地址A=0x240，则求得：S=0，O= 0x240

对于地址A=0x4050，则求得：S=？，O=？

分段的地址翻译

- 翻译下其他地址!



其他逻辑地址0x4050、0x6a50、0xdfa0对应物理地址？

分段技术总结

- 实现原理
  - 将程序按含义分成若干部分，即分段
  - ld从0开始编址每个段（链接速度会很快）：汇编时从0编址、链接时可不为0
  - 创建进程（分别载入各个段）时，建立进程段表
  - 内存仍用可变分区进行管理，载入段时需调分配算法
  - PC及数据地址要通过段表算出物理地址，到达内存
  - 进程切换时，进程段表也跟着切换



进程、内存、编译环境、编程思想被扭结在一起了，这正是操作系统的复杂之处！

分段技术优缺点分析

- 优点：

  符合人的习惯，程序员感觉舒服

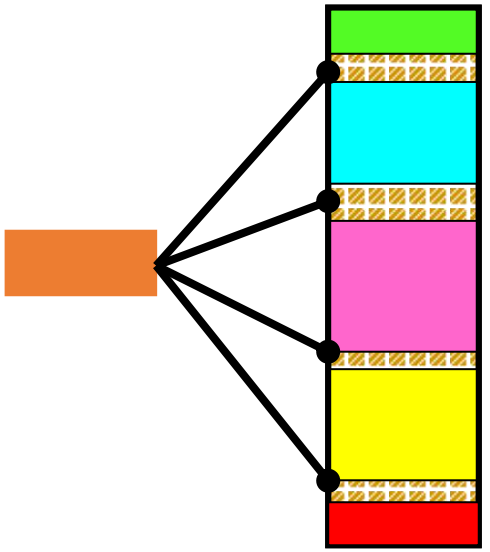
  - 不同的段有不同的含义，可区别对待
  - 每个段独立编址，编程容易（如果是一个大的一维地址空间，程序员一会儿就糊涂了！）（分治）

- 缺点：靠近了我们，必然会远离…

  空间低效

  - 空间预留；空闲空间很大却不能分配；内存紧缩
  - 著名的碎片概念：空闲的却用不上的空间

内部碎片　　外部碎片

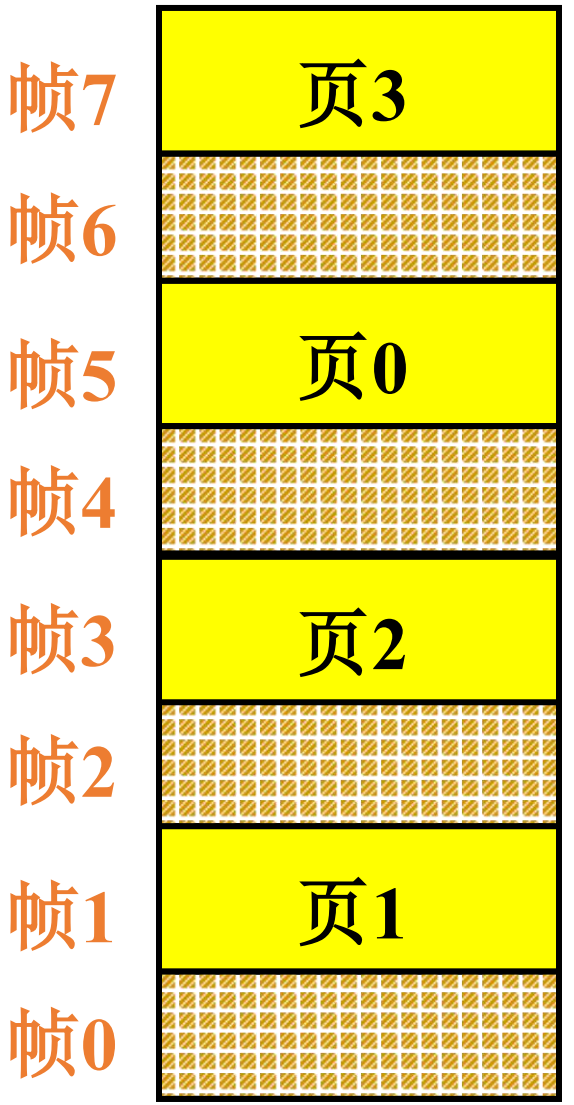


分页(Paging)!

思考：按需供给内存

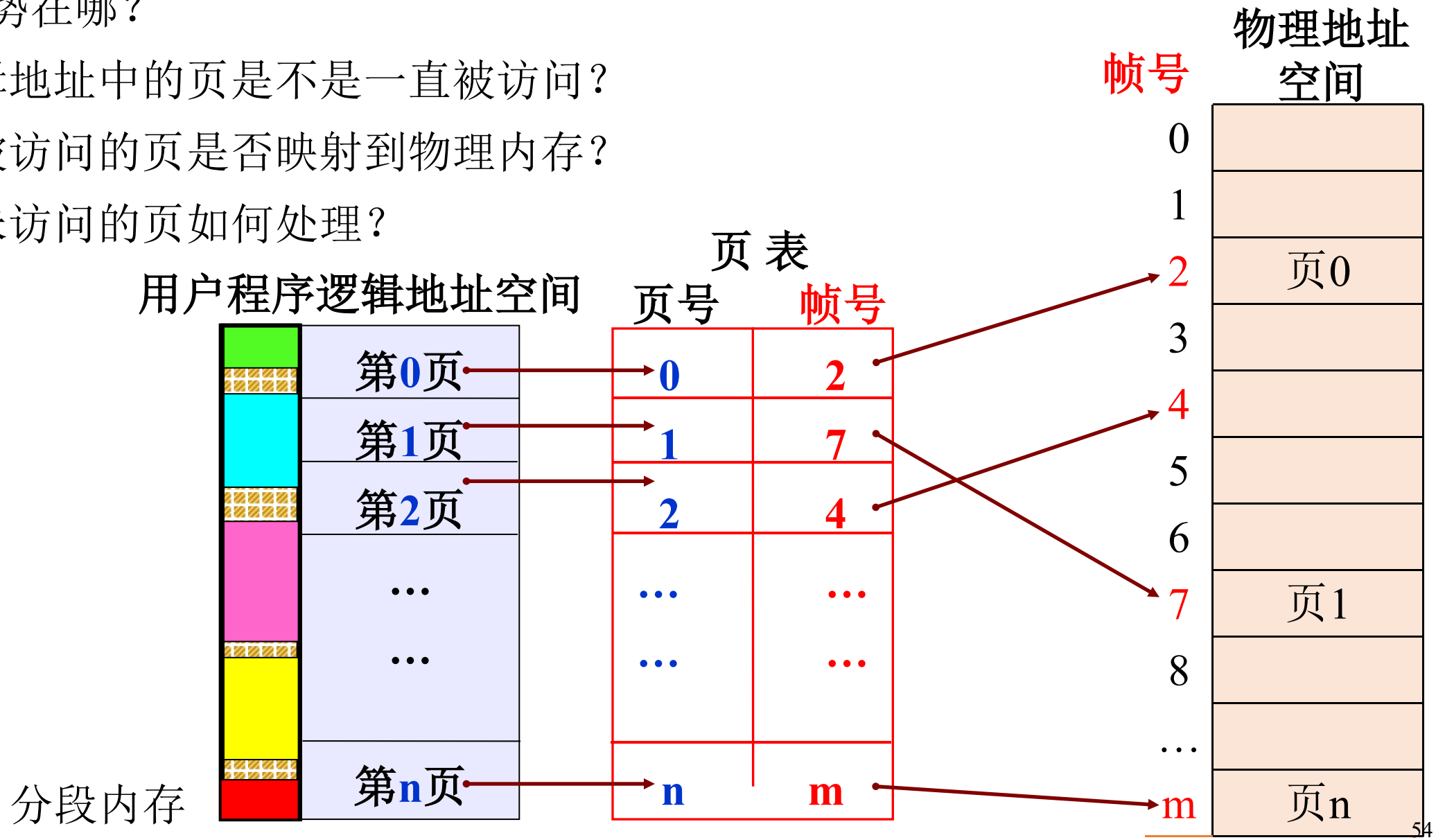
从连续到离散

- 只有吃到最后才能知道到底有多饿!
- 对于进程的堆栈段，只有运行完才知道嵌套深度
- 一次分配给一点（没有外部碎片，内部碎片有上界）
- 将面包切成薄片，将内存分成页



分页内存管理思想

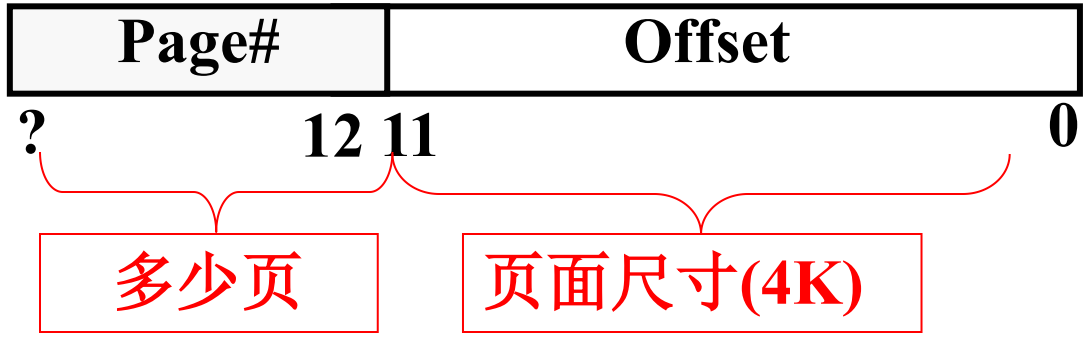
- 如此映射优势在哪？
- 1）程序逻辑地址中的页是不是一直被访问？
- 2）某些未被访问的页是否映射到物理内存？
- 3）长时间未访问的页如何处理？



分页机制中的页表

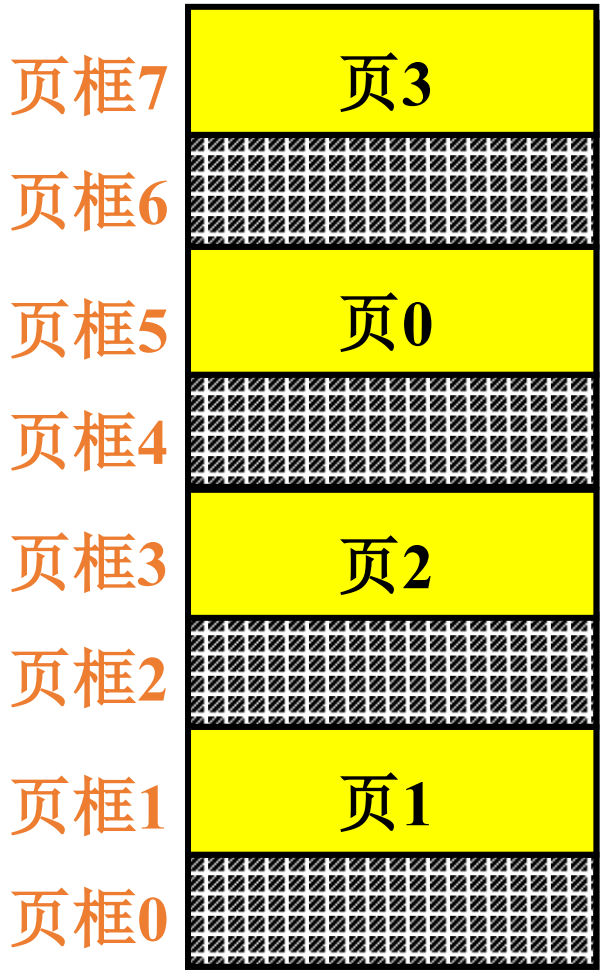
- 和分段类似，分页依靠页表结构

逻辑地址格式



进程页表

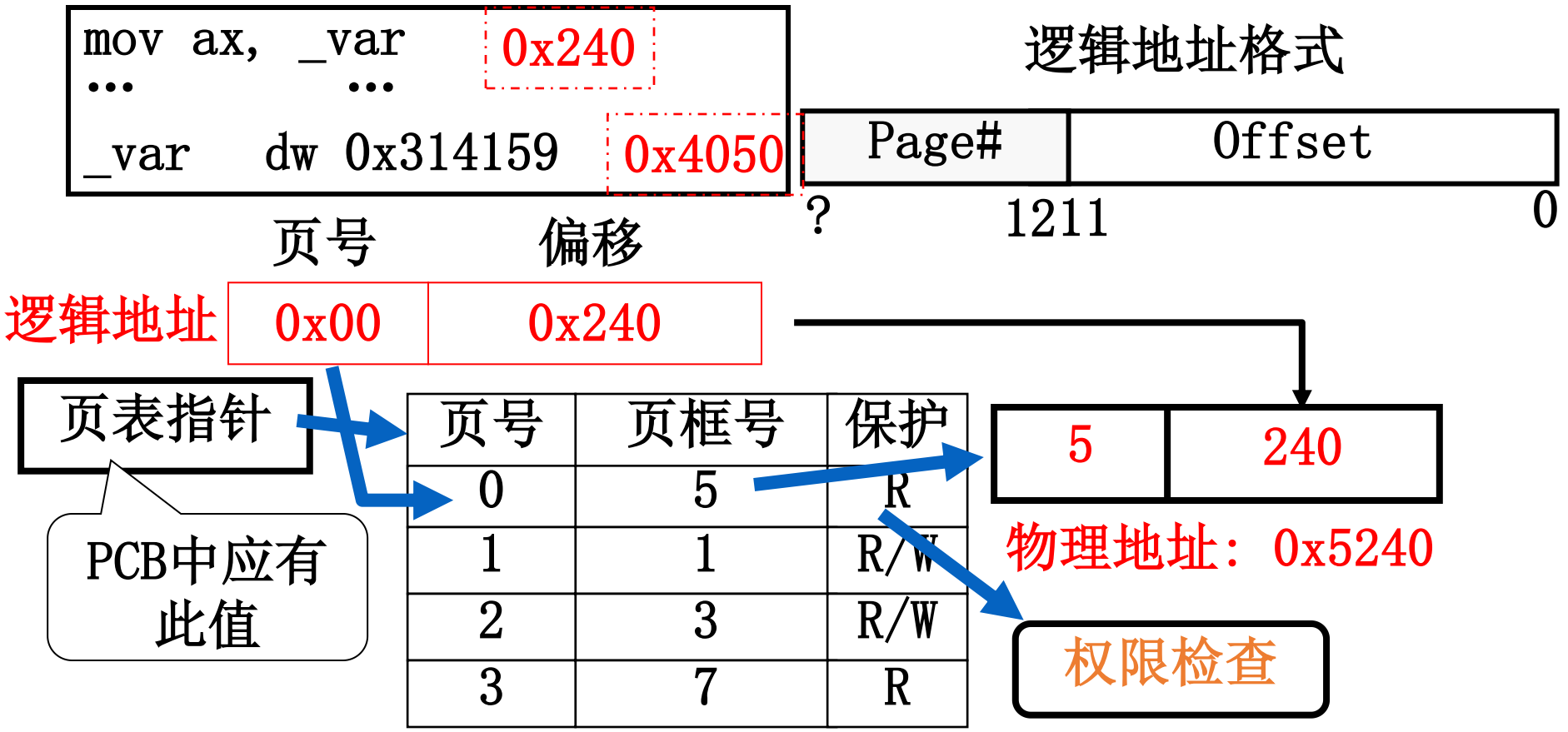
页号	页框号	保护
0	5	R
1	1	R/W
2	3	R
3	7	R



分页的地址翻译

- 一个实例!

物理地址0x39f0对应的逻辑地址？

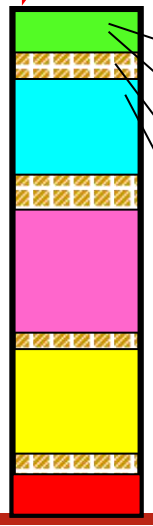


来考虑一些细节问题

- 和段表不一样，页表可能会很大!
- 页是用来解决碎片问题的=>页面尺寸应尽量小
- 页面尺寸通常为4K，32位机器，有2^{20}个页面
- 每条指令都需要查几次页表，因此查表效率很重要
- 如果页号不连续，需要查找，折半$\log_2(2^{20})=20$

页号	页框号	保护
0	5	R
1	1	R/W
3	3	R

应该是连续的!



页号	页框号	保护	有效
0	5	R	1
1	1	R/W	1
2			0
3	7	R	1

可以删掉!

页表项数据结构

可以删掉!

- 页表基本数据结构Page Table Entry(PTE)

页号	页框号	保护	有效
0	5	R	1
1	1	R/W	1
2			0
3	7	R	1

Intel x86结构的PTE

页框号(物理页号)ppn	保留	0	L	D	A	PCD	PWT	U	W	P
31-12 ?	11-9	8	7	6	5	4	3	2	1	0

- 由逻辑地址va找到物理地址，只有一级页表，每页4k

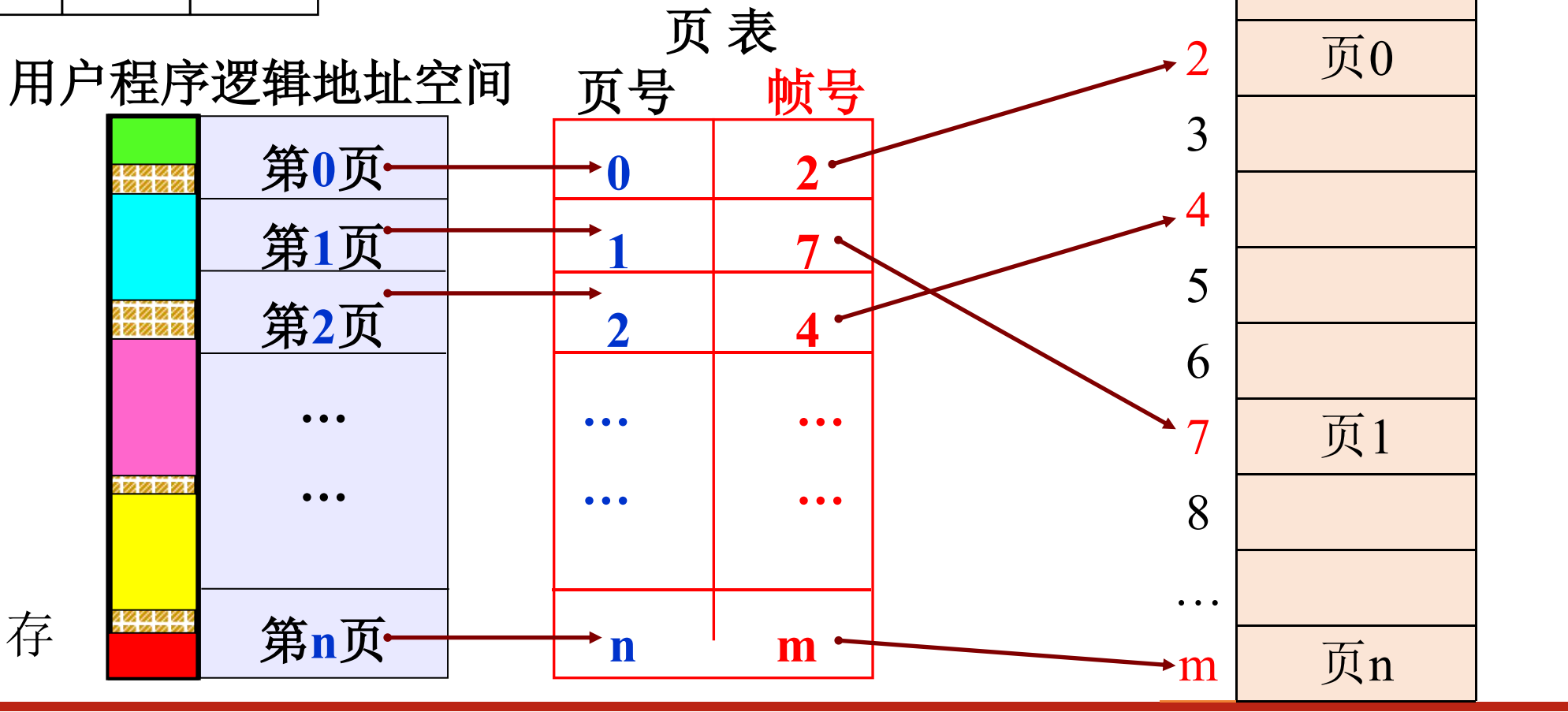
```
unsigned translate(unsigned va, int wr) {
  struct pte *pte = &page_table[va >> 12];//页号
  if(!pte->valid || (wr && !pte->writeable))
     throw address_fault;
   return (pte->ppn << 12) | (va & 0xfff); }
```

分页内存管理，逻辑地址空间范围

页号	页框号	保护	有效
0	5	R	1
1	1	R/W	1
2			0
3	7	R	1

逻辑地址和物理地址分开，其范围？

逻辑地址空间的页可以按需对应物理地址中的帧。由虚到实



分段内存

多级页表

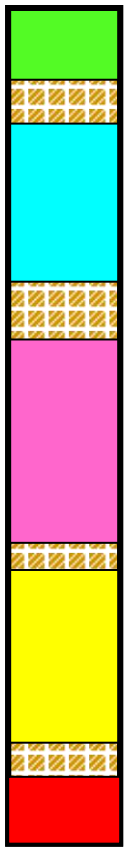
- 32位地址空间+ 4K页面+页号连续⇒2^{20}个页表项
- 2^{20}个页表项，每个4字节，都放在内存，要4M内存
- 系统中并发10个进程，需要40M内存

为什么每个进程需要独立一个页表？

- 实际上大部分逻辑地址根本不会用到

32位: 总空间[0，4G-1]!

- 引入多级页表，顶层页表常驻内存，不需要映射的逻辑地址不需要建立页表项



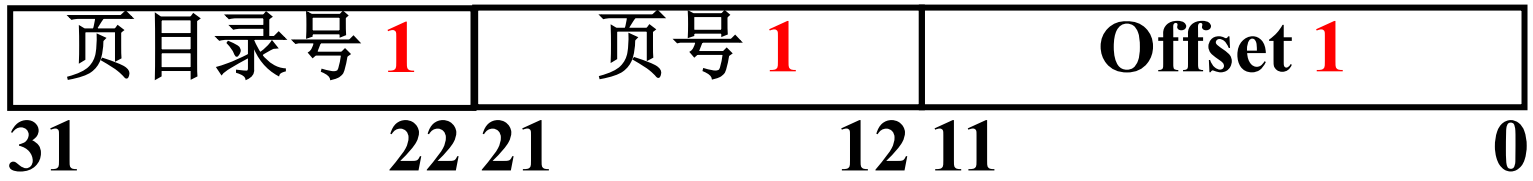
32位逻辑地址格式(多级页表)



多级页表

- 32位地址，4G内存：先按照4M分割，每个4M内再按照4K分割。

32位逻辑地址格式(多级页表)

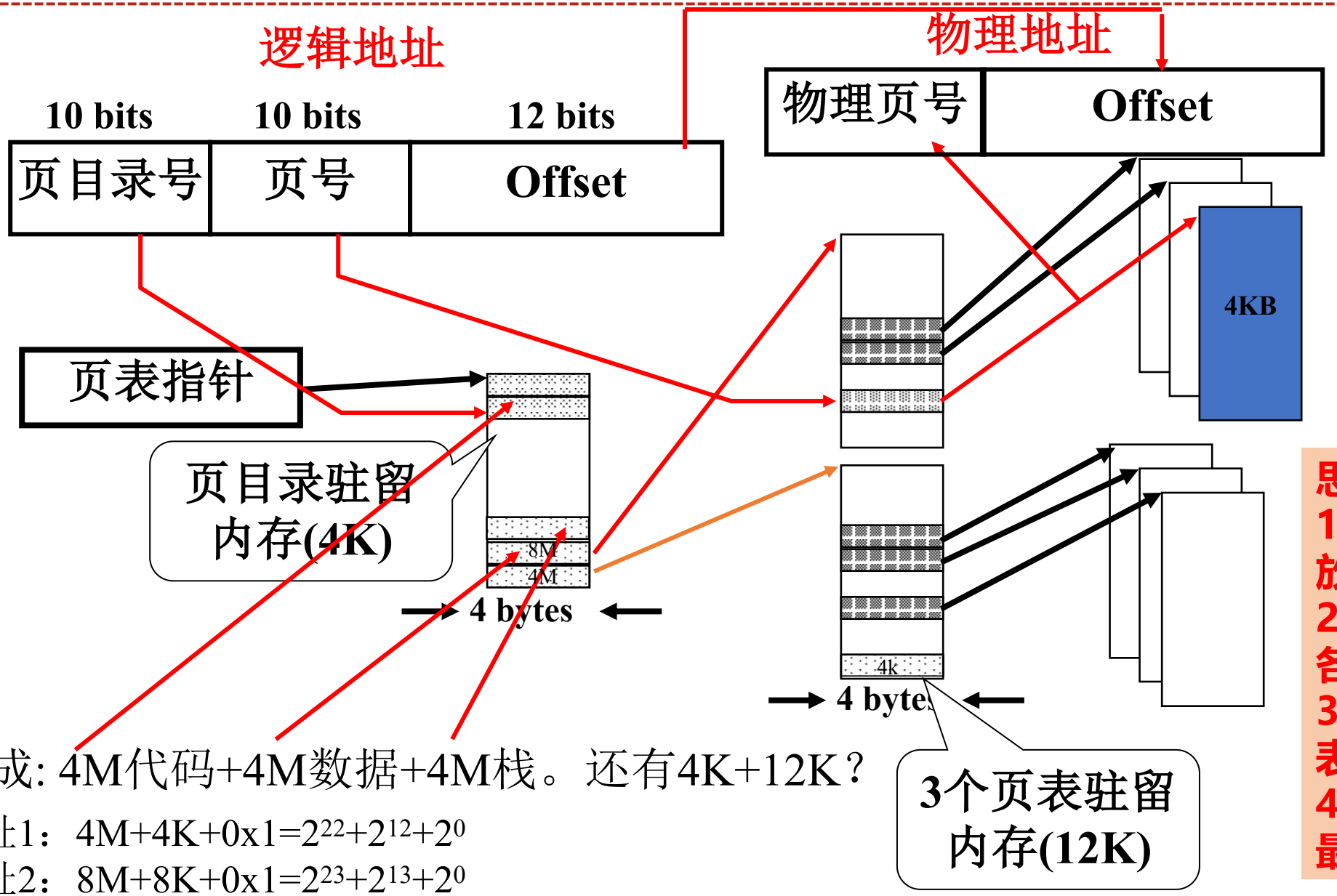


- 给定两个逻辑地址：

  逻辑地址1：$4M+4K+0x1=2^{22}+2^{12}+2^{0}$

  逻辑地址2：$8M+8K+0x1=2^{23}+2^{13}+2^{0}$

多级页表时的地址翻译



程序构成: 4M代码+4M数据+4M栈。还有4K+12K？

逻辑地址1： $4M+4K+0x1=2^{22}+2^{12}+2^{0}$

逻辑地址2： $8M+8K+0x1=2^{23}+2^{13}+2^{0}$

思考：

1、页目录、页表存放在哪里?

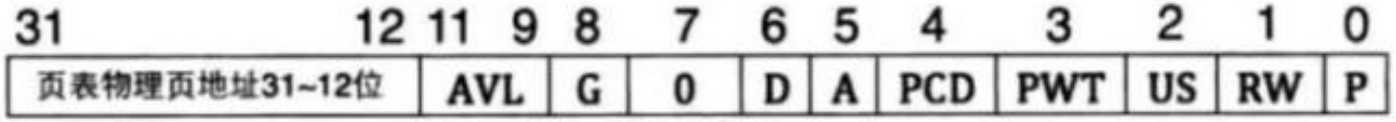
2、页目录、页表内各自存放什么内容?

3、一个页目录项代表多大的地址空间?

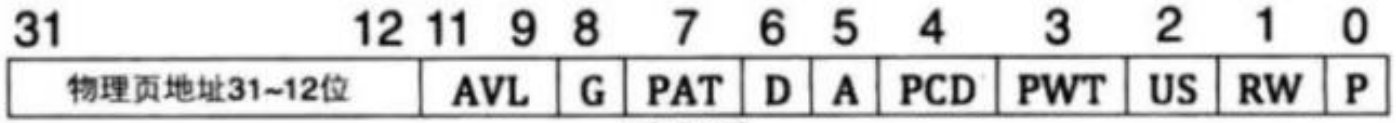
4、一个进程最少/最多消耗几个页表?

页目录项和页表项：以X86为例

PDE与PTE的结构



页目录项



页表项

P位：存在位，为1时表示该页在内存中

RW：读写位，为1时可读可写，为0是可读不可写

US：普通用户/超级用户位，为1时表示处于用户级，也就是3级特权级

PWT：页级通写位，为1表示此项采用通写方式，表示该页不仅是普通内存，还是高速缓存

PCD：页级高速缓存禁止位，为1表示该页启用高速缓存

A：访问位，为1表示该页被CPU访问过

D：脏页位，当CPU对一个页面执行写操作，此为被赋1

PAT：页属性表位，能够在页面一级的粒度上设置内存属性

G：全局位，为1表示该页在高速缓存TLB中一直保存

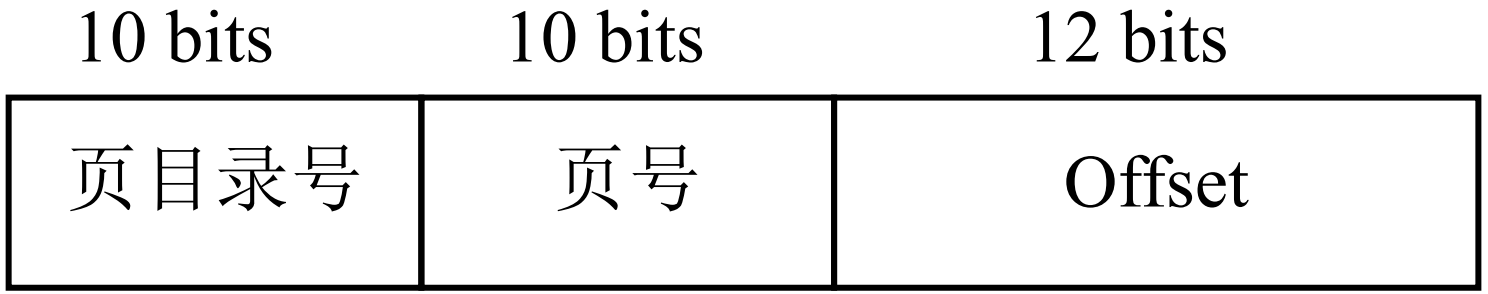
多级页表练习题

- 一个32位的内存管理系统使用了两级页表，其逻辑地址中，第22到31位是第一级页表的索引，12位到21位是第二级页表的索引，页内偏移占0到11位。

1、请问进程整个地址空间有多少字节？一页有多少字节？

2、如果每个页目录号下对应的所有二级页表全部分配，共占用多少页面？这些二级页表的存储需要多少字节？

3、逻辑地址0xC030f000对应的页目录号和页号分别是多少？



（注意B代表字节，一个32位地址占4字节）

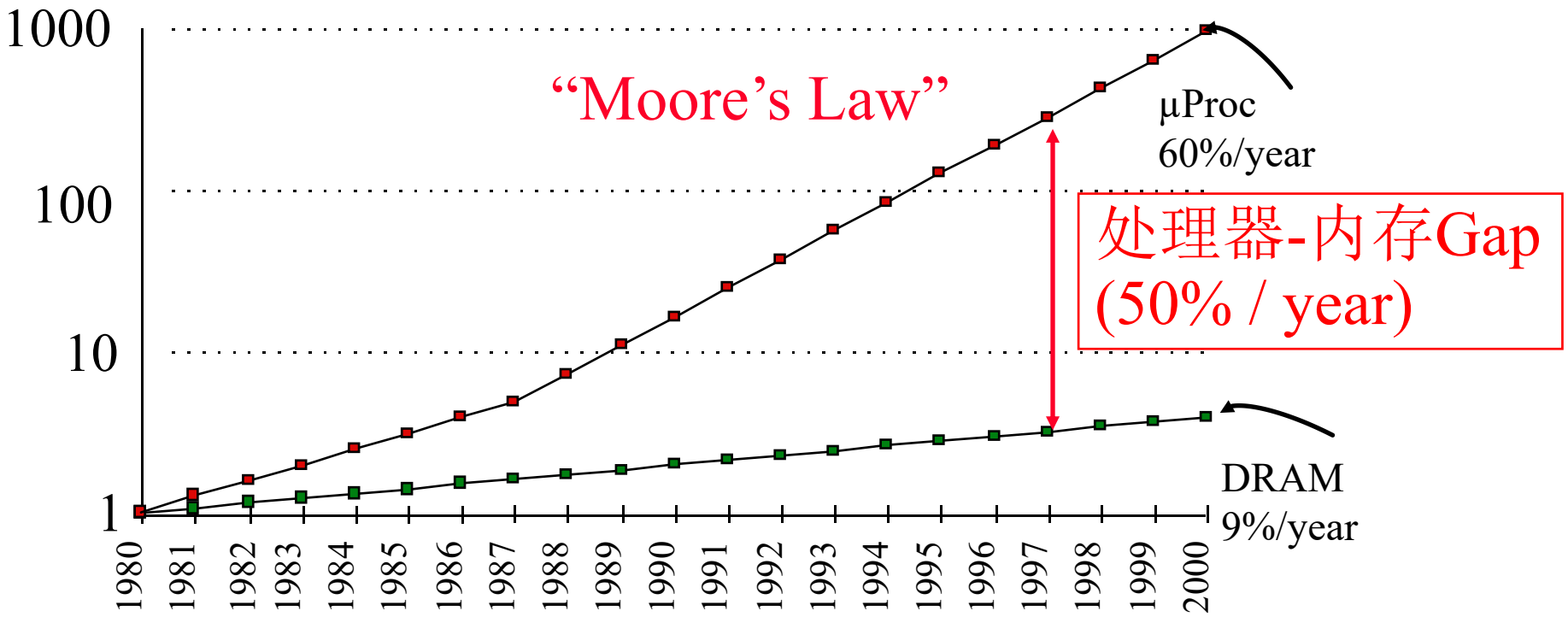
多级页表使得地址翻译效率更低

一次地址访问

- 1级页表访存2次，速度下降50%
- 2级页表访存3次，速度下降到33%
- 3级页表访存4次，速度下降到25%

访问页表计算物理地址，从物理地址取内容

需要注意的事实：内存相比CPU本来就很慢！



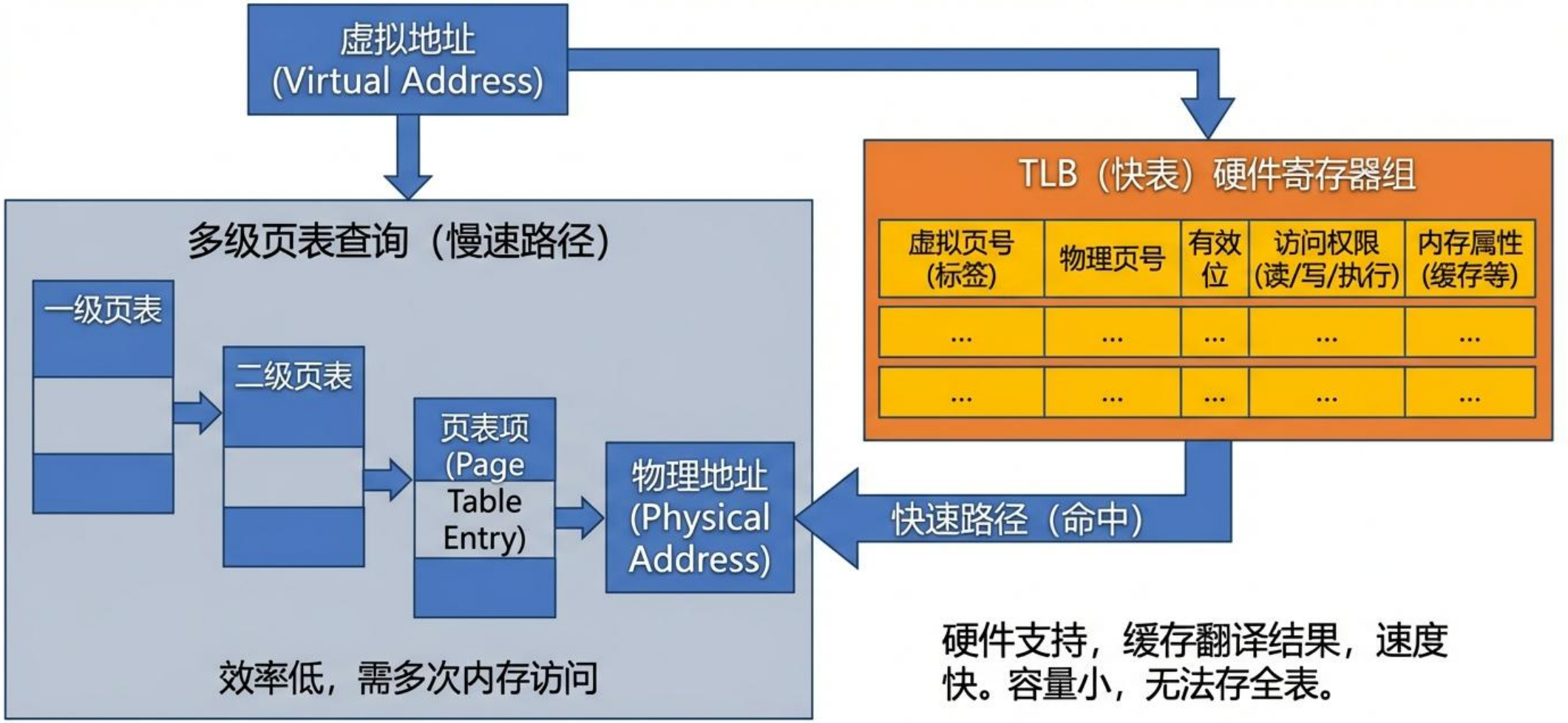
提高地址翻译的效率

- 多级页表的地址翻译效率很低，要提高效率
- 提高效率的基本想法: 硬件支持
- 要很快：这个硬件放在哪里？ **寄存器**
- 页表小=>寄存器可行，但如果页表很大呢?
- TLB(Translation Look-aside Buffer)是一组关联快速寄存器组

可参考《步步惊芯——软核处理器内部设计分析》

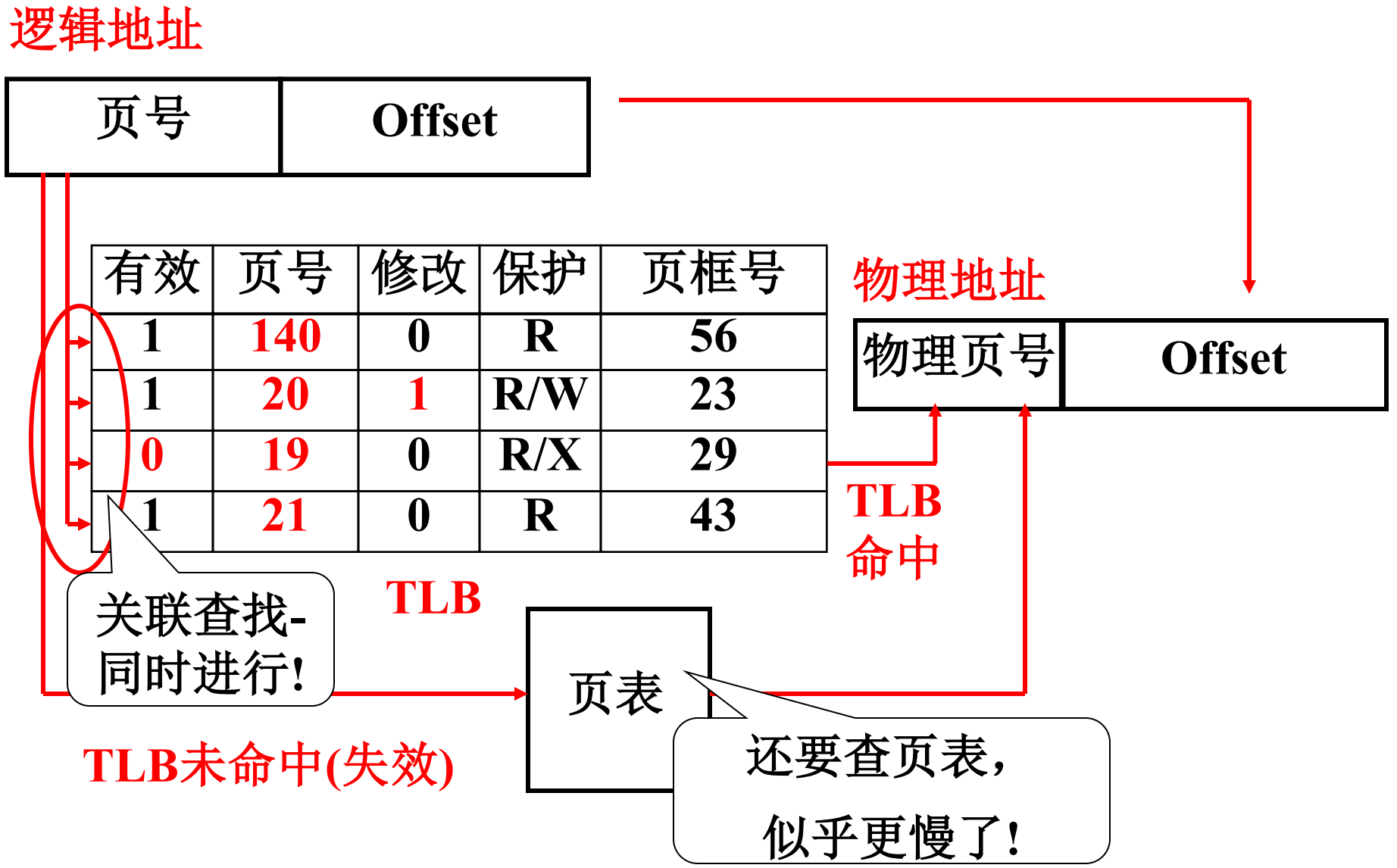
有效	页号	修改	保护	页框号
1	140	0	R	56
1	20	1	R/W	23
0	19	0	R/X	29
1	21	0	R	43

TLB的缓存



TLB加速：多级页表慢，硬件寄存器TLB缓存'虚拟页号->物理页号'映射及控制位，提升效率。

采用TLB后的地址翻译



TLB得以发挥作用分析

- TLB命中时效率会很高，未命中效率会降低，平均后仍表现良好。用数字来说明（假设使用一级页表）：

有效访问时间 = HitR×(TLB+MA) + (1-HitR)×(TLB+2MA)

命中率!

内存访问时间!
假设100ns

TLB时间!
假设20ns

有效访问时间=80%×(20ns+100ns) + 20%×(20ns+200ns)=140ns

有效访问时间=98%×(20ns+100ns) + 2%×(20ns+200ns)=122ns

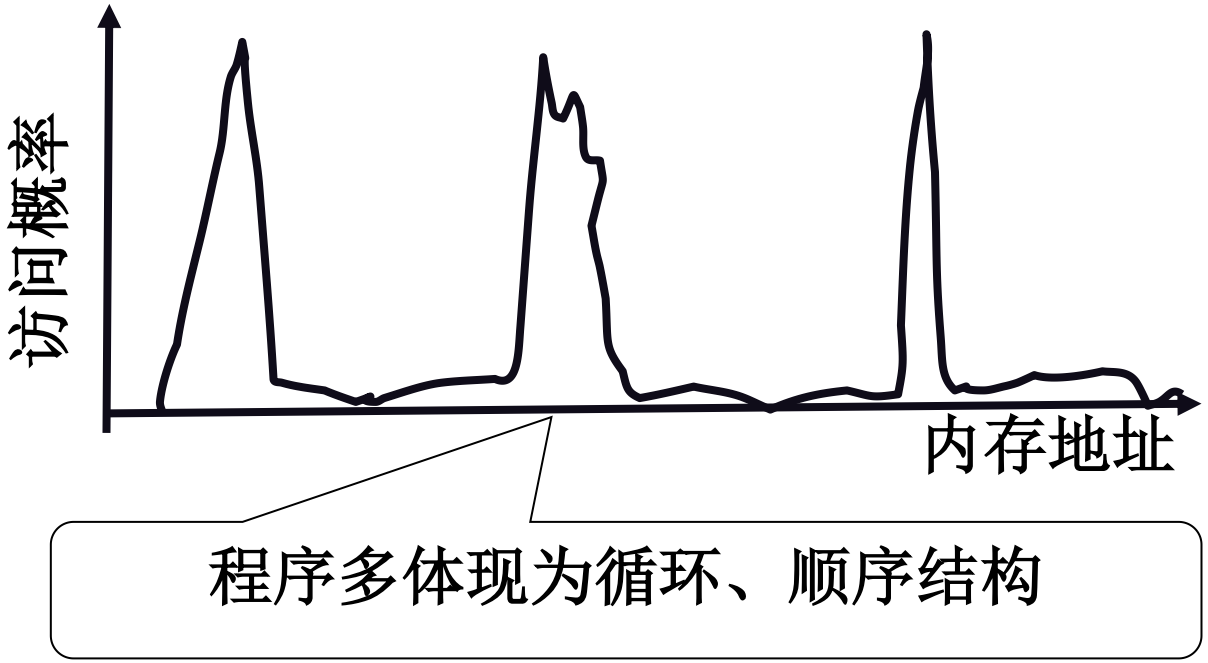
平均加快了13%!

TLB要想发挥作用，命中率应尽量高

TLB越大越好，但TLB价格昂贵，通常[64, 1024]

为什么TLB条目数在64-1024之间?

- 相比2^{20}个页，64很小，为什么TLB就能起作用?
- 程序的地址访问存在局部性
- 空间局部性(Locality in Space)





局部性又是计算机
的一个基本特征

TLB条目少，页表项多=>TLB动态变化

- 如果TLB未命中，可将查到的页表项载入TLB
- 如果TLB已经满了，需要选择一个条目来替换
- 有些时候希望某些条目固定下来（如内核代码），某些TLB的设计有这样的功能，不被选择替换
- 进程切换后，所有的TLB表项都变为无效（flush）
- 如果进程马上又切换回来，则这种策略就很低效。有的TLB设计中条目项保存ASID（Address-space identifier），此时不需要flush。

分页技术总结

实现机理

- 逻辑地址空间和内存都分割大小相等的片（页和页框）
- 每个进程用页表（多级、反向等）建立页和页框的映射
- 进程创建时申请页，可用表、位图等结构管理空闲页
- 逻辑地址通过页表算出物理地址，到达内存
- 进程切换时，页表跟着切换
- 优点：靠近硬件，结构严格，高效使用内存
- 缺点：不符合程序员思考习惯

分页更适合于自动化（硬件实现）！

随堂习题

- 例1：已知某分页系统，主存容量为64K，页面大小为1K，对一个4页大的作业，第0、1、2、3页被分配到内存的2、4、6、7块中。求：将十进制的逻辑地址1023、2500、4500转换成物理地址。

随堂习题

- 设有8页的逻辑空间，每页有1024字节，它们被映射到32块的物理存储区中，那么逻辑地址的有效位是__位，物理地址至少是__位。

段页结合内存管理

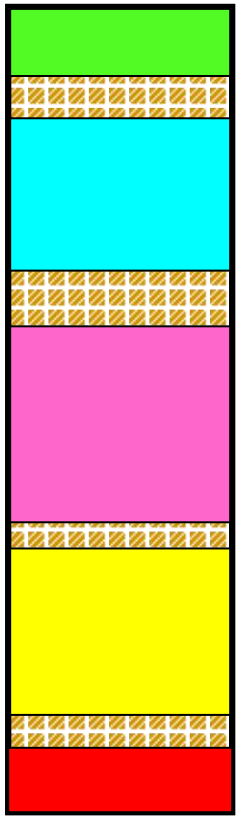
段、页结合!

操作系统的可执行文件格式

- 每种操作系统都规定了可以运行文件格式（可执行文件）；
- ELF\COFF\OUT等

Linking View
ELF header
Program header table *optional*
Section 1
...
Section *n*
...
...
Section header table

Execution View
ELF header
Program header table
Segment 1
Segment 2
...
Section header table *optional*



让段面向用户、让页面向硬件



段号+偏移(cs:ip)

段号	基址	长度	保护
0	0x4000	0x0800	R
1	0x4800	0x1400	R/W
2	0xF000	0x1000	R/W
3	0x0000	0x3000	R

逻辑地址

页号 | 偏移

物理地址

物理页号 | 偏移

页框号	保护
5	R
1	R/W
3	R/W
7	R

常称为线性地址，以示区别

段页式内存管理的基本视图



cs:ip

逻辑地址

4G

操作系统段

用户代码段

用户数据段

用户栈段

0

虚拟地址空间

0x00345008

线性地址

物理地址

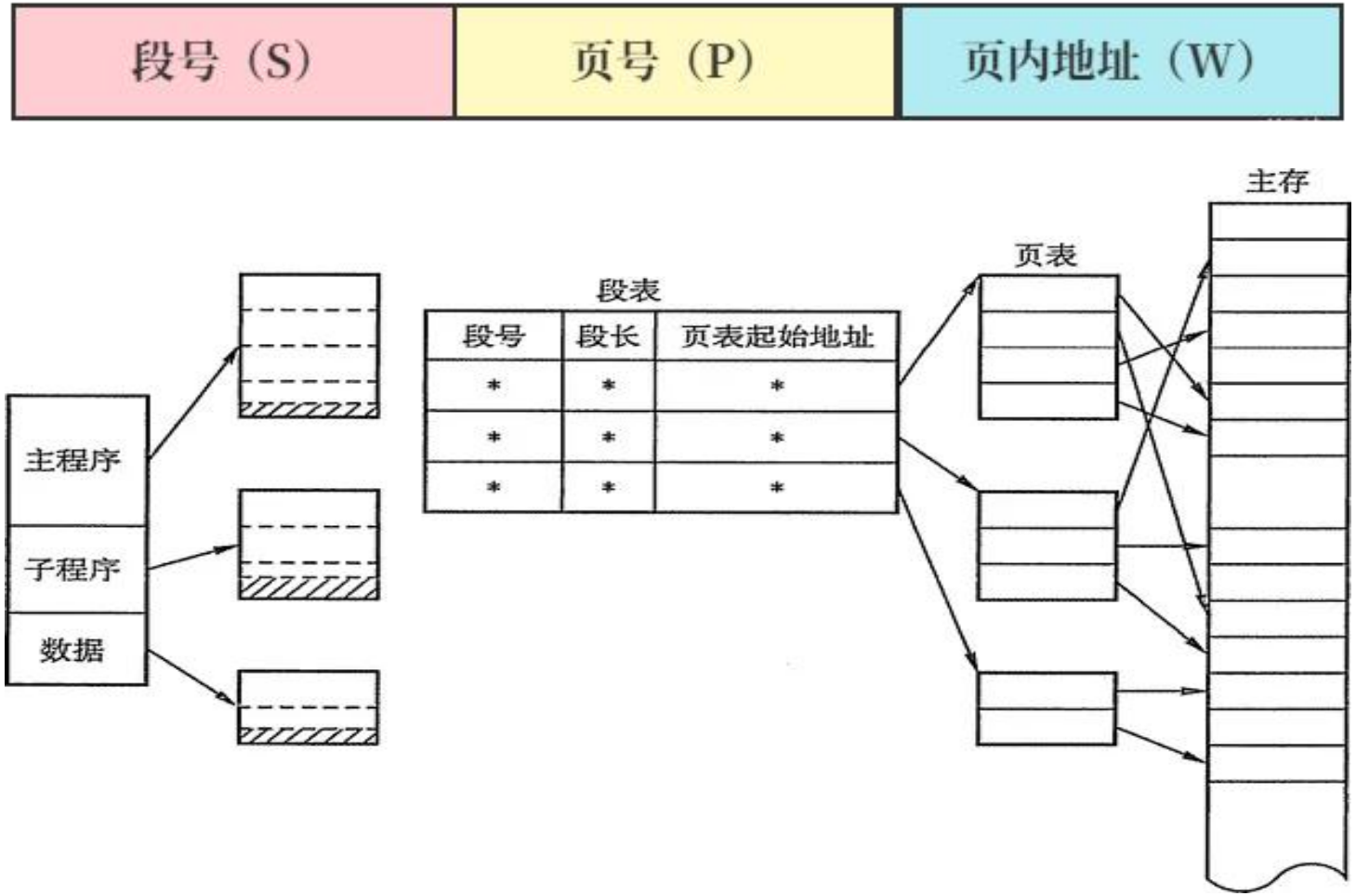
0x7008

内存

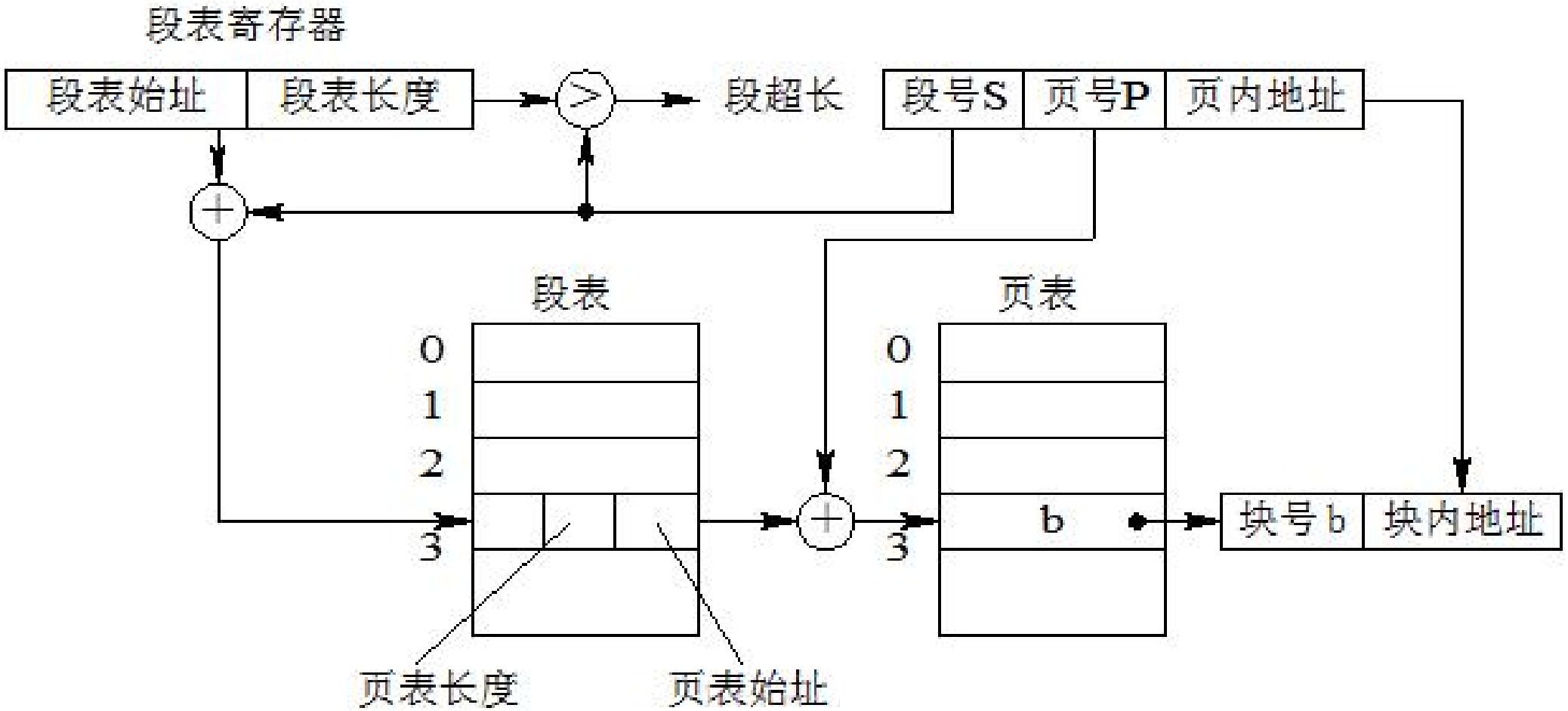
对用户是透明的

段页式管理

- 结合了页式存储和段式存储的方法，采用了分段的方式，每一段作业独立编程，再把段分成若干个页面。



段页式管理


段表寄存器
段表始址
段表长度
>
段超长
段号S
页号P
页内地址
段表
0
1
2
3
页表
0
1
2
3
b
块号 b
块内地址
页表长度
页表始址

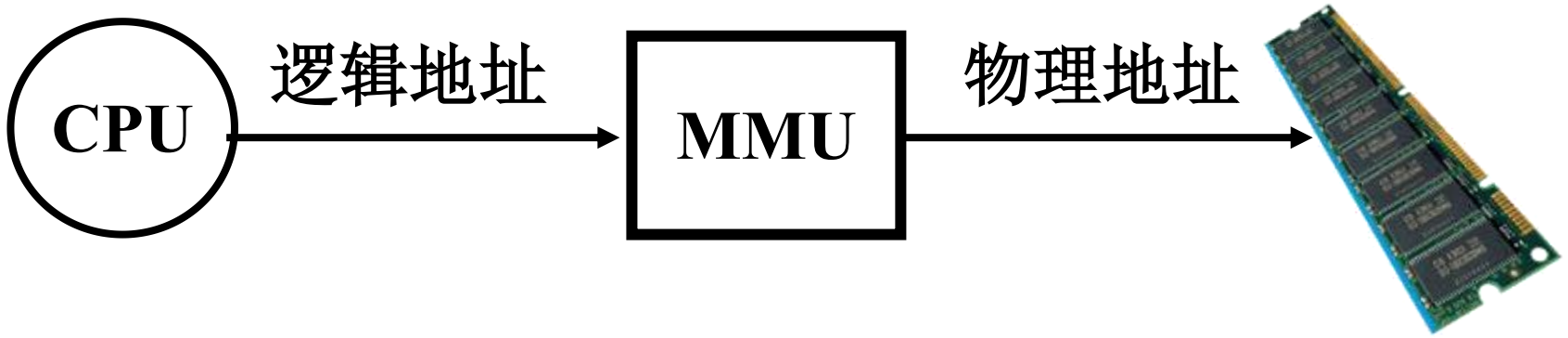
段页结合技术总结

- 实现机理
  - 程序的段划分的是线性地址空间（如0-4G）
  - 线性地址空间和内存被分割大小相等的片（页和页框）
  - 进程用页表建立页和页框的映射
  - 进程创建申请段（线性地址空间），段申请页（物理内存）
  - 逻辑地址通过段表加页表算出物理地址，到达内存
  - 进程切换时，段表和页表都跟着切换
  - 优点：符合程序员习惯，并可高效利用内存
  - 缺点：复杂，访问一次地址需要查表好多次...

Intel x86的内存

操作系统课讲硬件，大家可能觉得奇怪

- 段页结合时，进行一次地址翻译需要:
  - (1)找到段表；(2)查段表；(3)找TLB；(4)找到页目录表；(5)查找页目录项；(6)找到页表；(7)查找页表；(8)形成物理地址；(9)需要段越界检查；(10)需要进行段保护权限检查；(11)需要进行页保护权限检查...
- 如此多的事情都用软件实现，其效率会很低

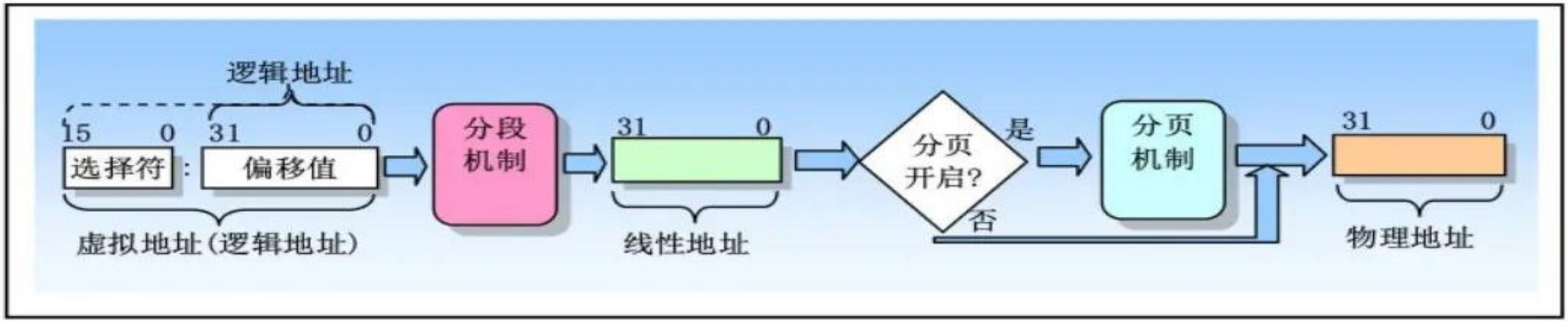


Intel x86的内存管理硬件!

- 实模式，内存地址20位，1M。
- 保护模式，地址32位，4G。
- 寄存器CR0.PE位=1进入实模式。

X86地址变换

- 保护模式开启分页机制；
- 实模式和保护模式分段管理有区别；
- 实模式段寄存器中存储段基址；
- 保护模式段选择器存储段描述符表中的索引；



嵌入式软件多数运行在没有（未开启）分页机制的处理器上

Intel x86的内存管理硬件!

- 实模式称为：段寄存器(CS, DS, ES, SS)
- 保护模式称为：段选择器（6个）

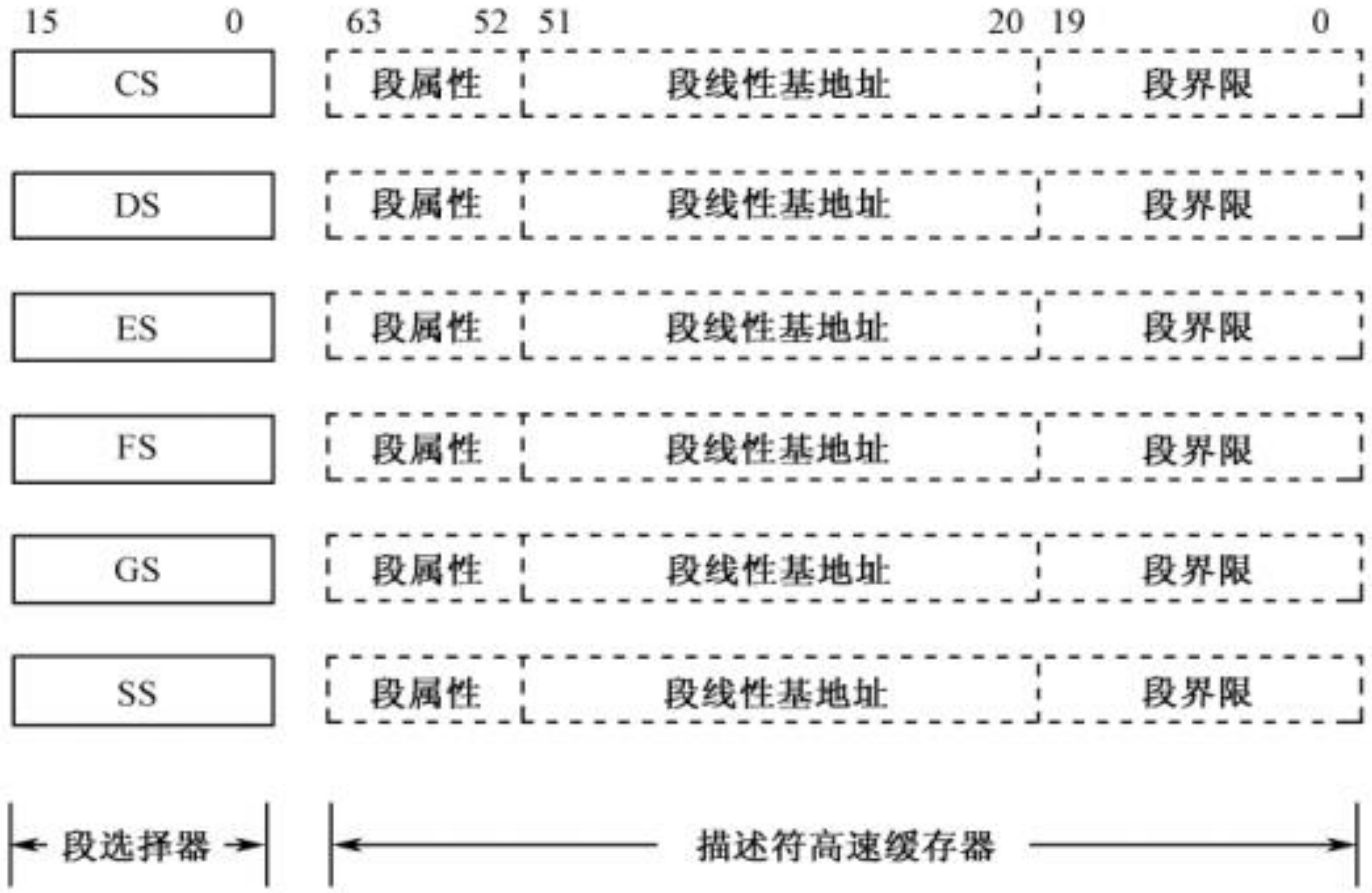


图 11-9　32 位处理器内的段寄存器

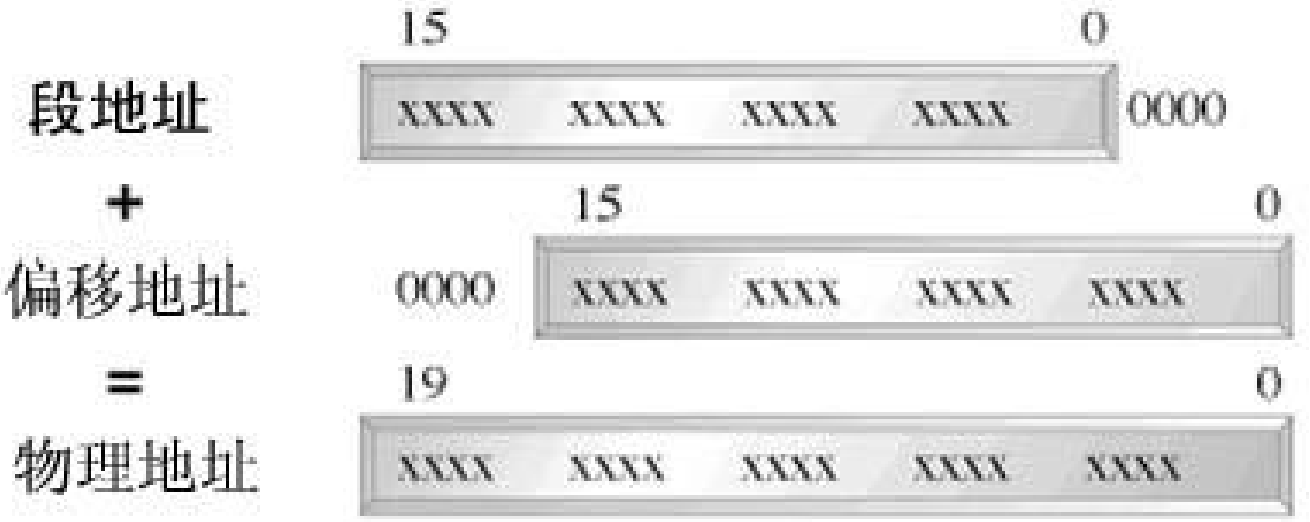
```
(gdb) info registers
rax            0x21                33
rbx            0x0                 0
rcx            0x1                 1
rdx            0x0                 0
rsi            0x5555555592a0      93824992252576
rdi            0x7fffffffdf20      140737488346912
rbp            0x7fffffffe490      0x7fffffffe490
rsp            0x7fffffffe480      0x7fffffffe480
r8             0x0                 0
r9             0x7fffffffe34c      140737488347980
r10            0x0                 0
r11            0x246               582
r12            0x7fffffffe5c8      140737488348616
r13            0x5555555551e2      93824992236002
r14            0x555555557db8      93824992247224
r15            0x7ffff7ffd040      140737354125376
rip            0x5555555551a8      0x5555555551a8 <foo+63>
eflags         0x206               [ PF IF ]
cs             0x33                51
ss             0x2b                43
ds             0x0                 0
es             0x0                 0
fs             0x0                 0
gs             0x0                 0
k0             0x7200012           119537682
k1             0x201088            2101384
k2             0x0                 0
k3             0x0                 0
k4             0x0                 0
k5             0x0                 0
k6             0x0                 0
k7             0x0                 0
```

Intel x86的内存管理硬件!

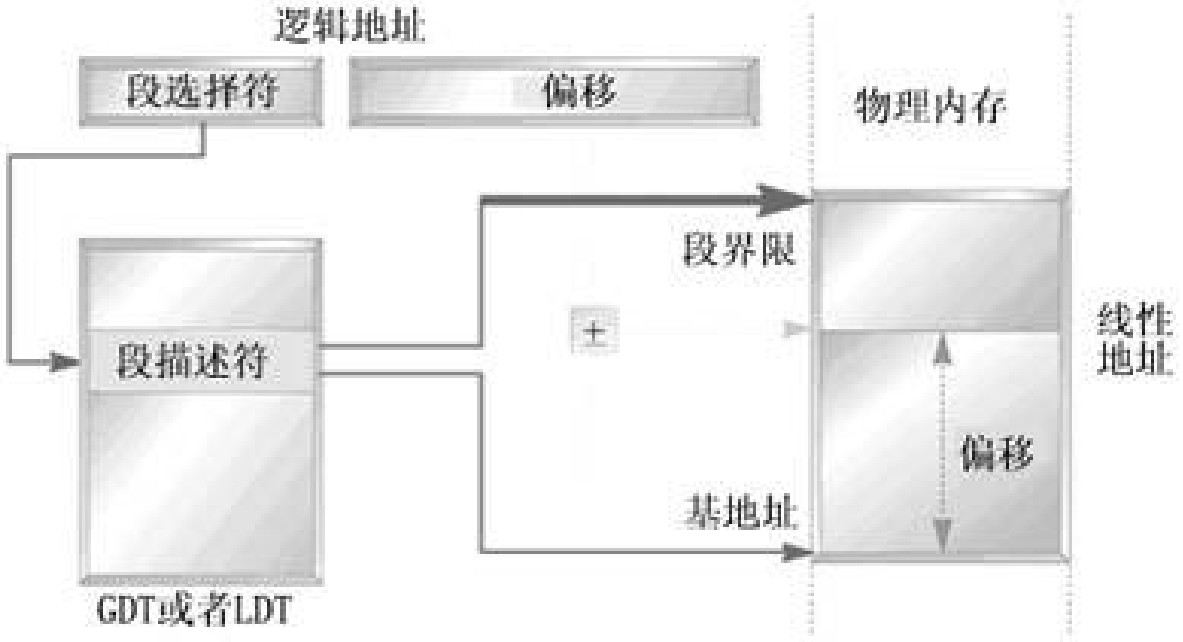
- 实模式下：(CS,DS,ES, SS)
- mov cx,0x2000
- mov ds,cx
- mov [0xc0],ax
- 基地址0x20000（左移4位，4字节对齐）
- **实际写入物理地址0x200c0**
- 线性地址与物理地址一致（实模式不采用分页机制）

实模式地址与保护模式地址

- 实模式：地址空间1MB



- 保护模式：地址空间4GB



段选择符的确定

段选择符确定步骤：从源代码到执行

1 第一步：编译与预处理

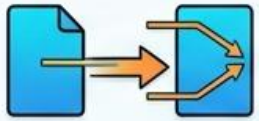
源文件 (.c/.asm) → 编译器生成目标文件 (.obj)



指令类型

1. **段内跳转：** 生成“近”（Near）指令，只有偏移地址。

   Near: (Near) 指令地址。



2. **跨段跳转/调用：** 生成“远”（Far）指令，带有符号引用。



2 第二步：链接与逻辑映射

链接器合并文件 **分配逻辑段号（文件内部索引）**

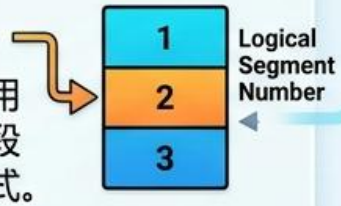
1. **合并同名段：** 合并代码、数据等。
2. **解析符号：** 远调用被填充为（逻辑段号，偏移）的形式。

注： “逻辑段号”还不是最终段选择符。



分配逻辑段号(文件内射)

1. **合并同名：** 合并代码等。
2. **解析符号：** 远调用被填充为（逻辑段号，偏移）的形式。

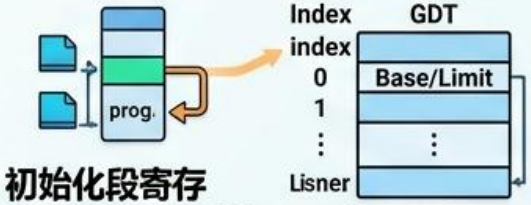


注： ‘逻辑段号’还不是最终段选择符。

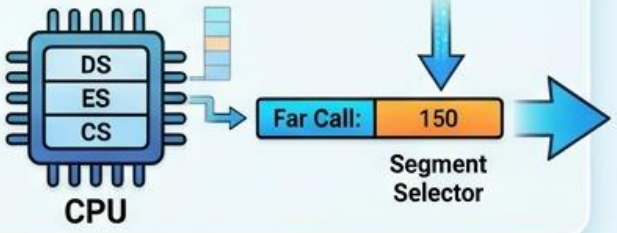
3 第三步：加载与重定位

加载器与操作系统（OS）关键重定位

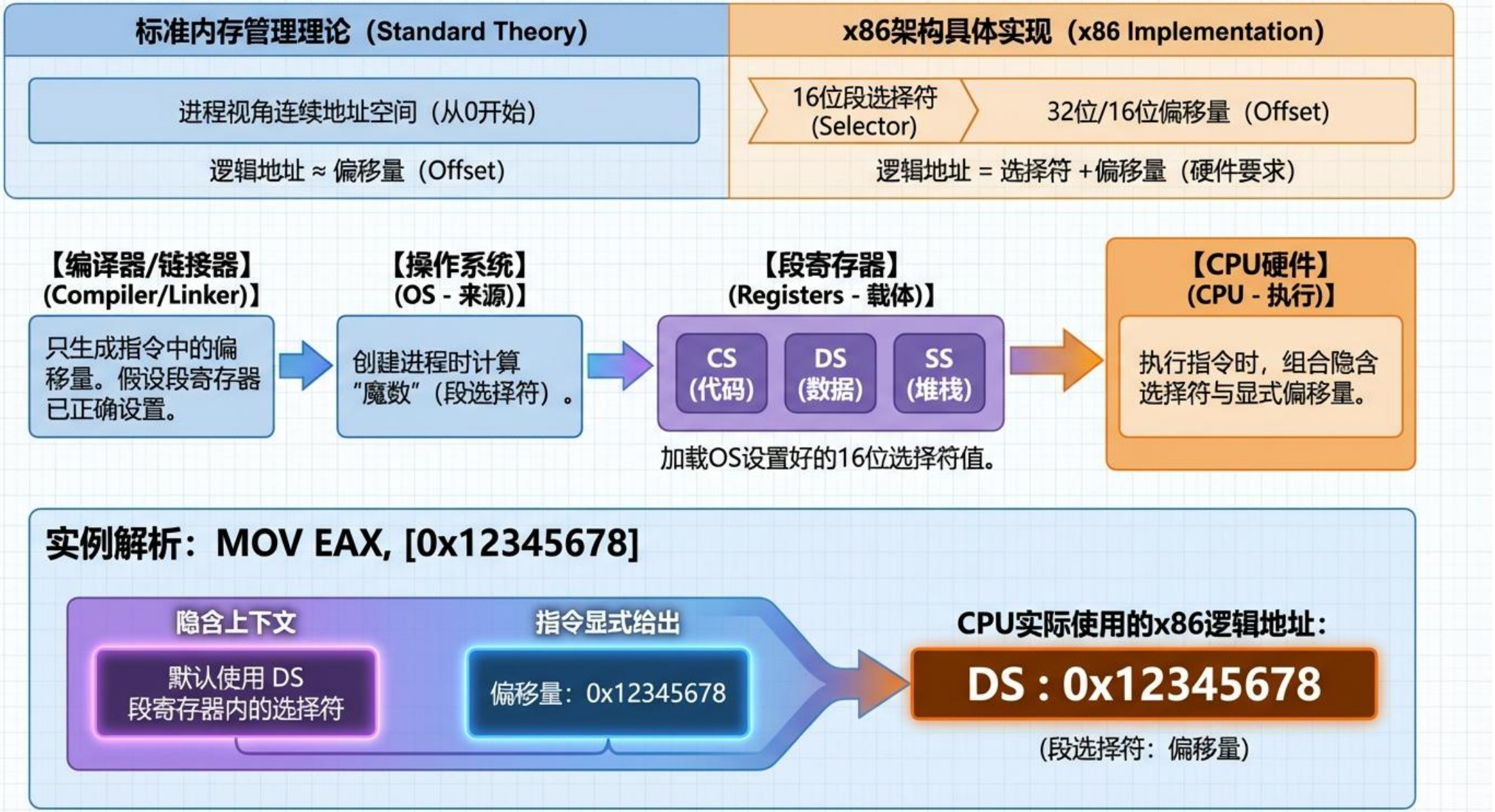
1. **构建描述符表：** 在GDT或LDT中创建描述符，映射“逻辑段号”到真实实段选择符。
2. **重定位指令：** 操作系统将远调用中的“逻辑段号”替换为真实段选择符。



3. **初始化段寄存器：** 将主数据段和代码段入口的段选择符加载到DS，ES，CS:IP。



段选择符的确定

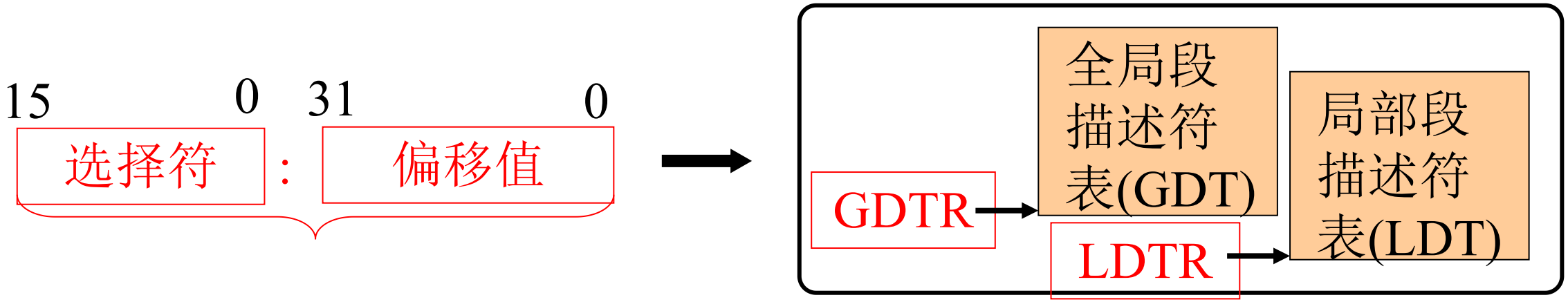

标准内存管理理论（Standard Theory）
x86架构具体实现（x86 Implementation）
进程视角连续地址空间（从0开始）
逻辑地址 ≈ 偏移量（Offset）
16位段选择符 (Selector)
32位/16位偏移量（Offset）
逻辑地址 = 选择符 +偏移量（硬件要求）
【编译器/链接器】 (Compiler/Linker)】
只生成指令中的偏移量。假设段寄存器已正确设置。
【操作系统】 (OS - 来源)】
创建进程时计算“魔数”（段选择符）。
【段寄存器】 (Registers - 载体)】
CS (代码)
DS (数据)
SS (堆栈)
加载OS设置好的16位选择符值。
【CPU硬件】 (CPU - 执行)】
执行指令时，组合隐含选择符与显式偏移量。
实例解析：MOV EAX, [0x12345678]
隐含上下文
默认使用 DS 段寄存器内的选择符
指令显式给出
偏移量：0x12345678
CPU实际使用的x86逻辑地址：
DS : 0x12345678
(段选择符：偏移量)

Intel x86的分段硬件

段号	基址	长度	保护
0	0x4000	0x0800	R
1	0x4800	0x1400	R/W
2	0xF000	0x1000	R/W
3	0x0000	0x3000	R

- 选择符: CS, SS, DS, ES, FS, GS(16位寄存器)
- 偏移值: EIP, ESP, ESI, EDI(32位寄存器)
- GDT: 全局段表(进程共享,OS各段,进程LDT/TSS入口)
- LDT: 局部段表(各进程独有，描述进程各段)
- GDTR和LDTR: 32位段表基址(线性地址)+16位段表长度。需用特权指令LLDT和SLDT等。
- TSS: 任务状态段，保存任务各寄存器值。通过TR寄存器访问

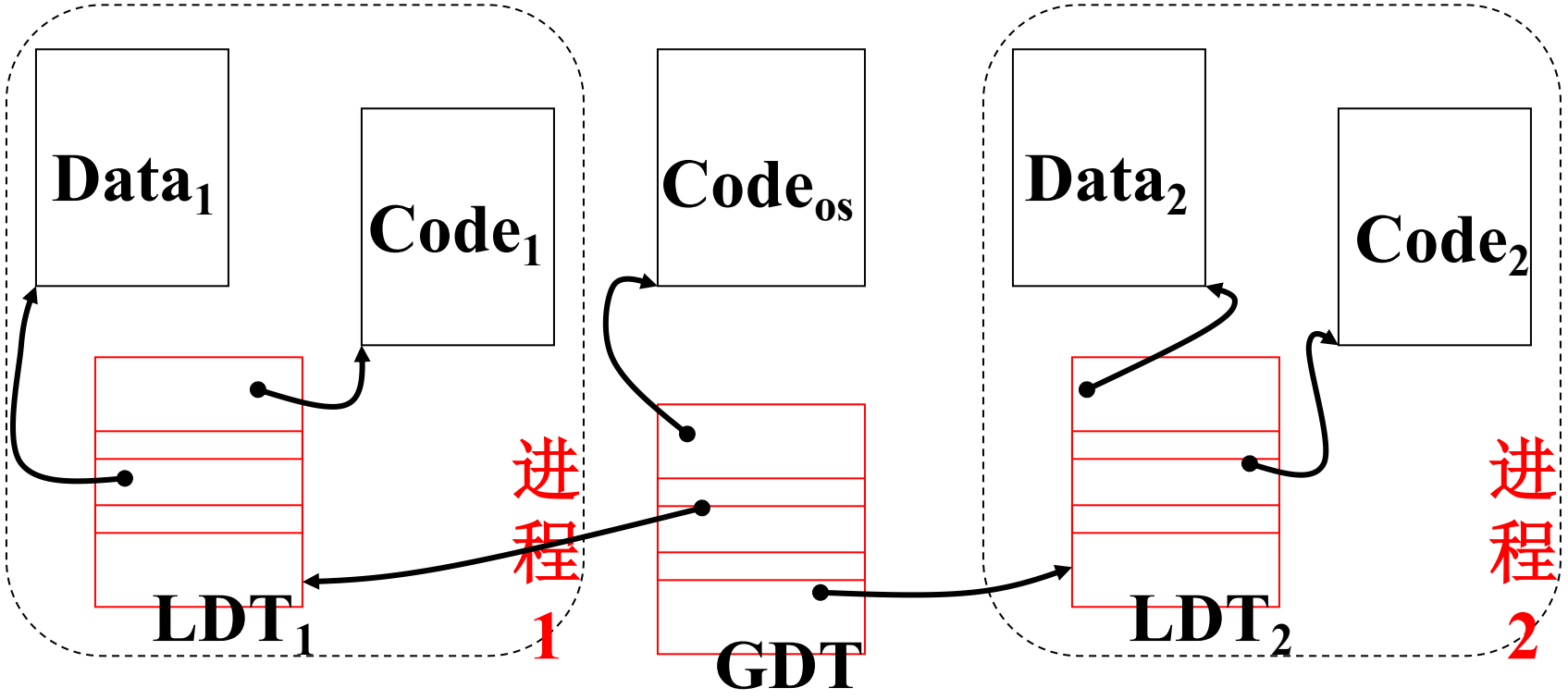
Intel x86的分段硬件



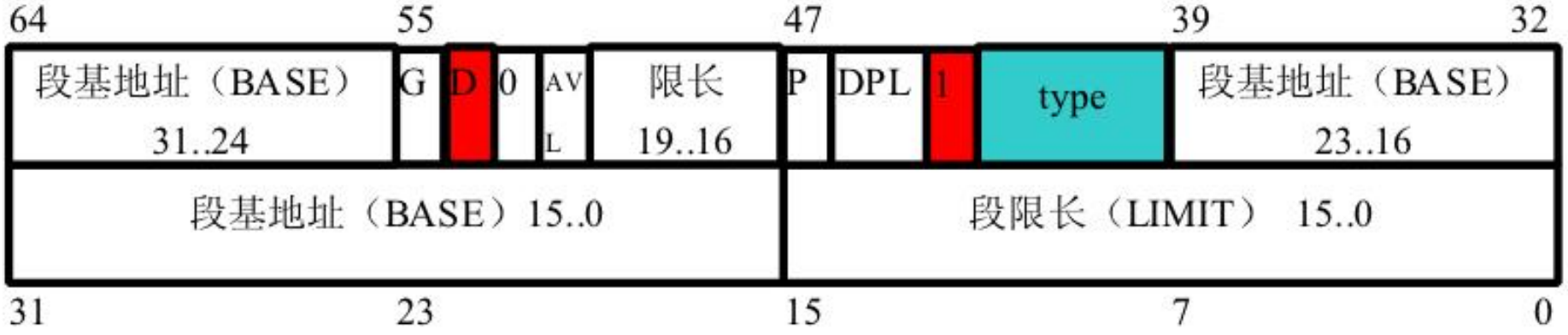
逻辑地址(如PC-CS：IP）→ 线性地址

- 每个进程需要有自己的段表LDTR
- 操作系统有自己的段表GDTR，同时还要记录访问每个进程的段表入口

Intel x86的分段硬件 - LDT、GDT

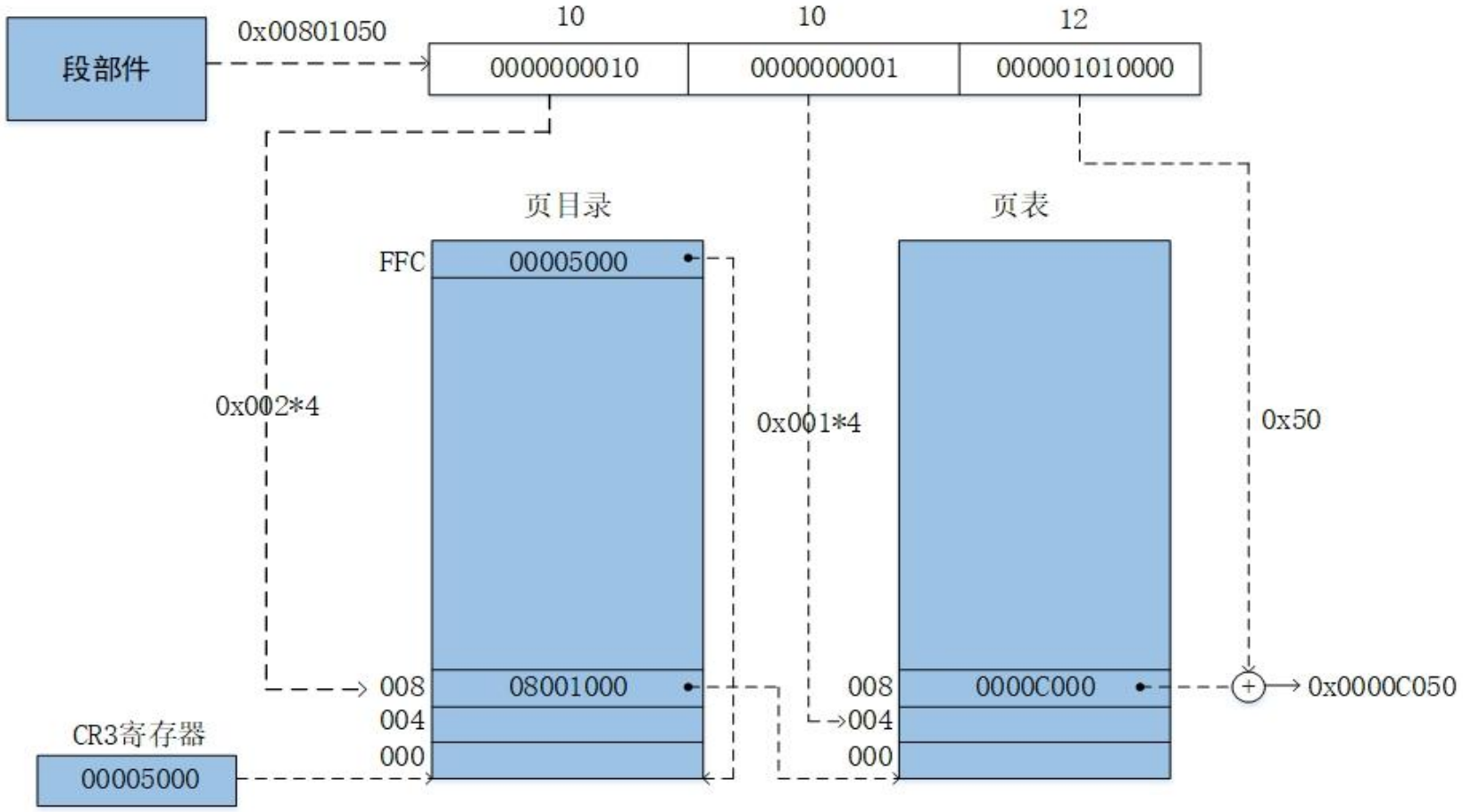


段描述符：
LDT(GDT)
中的表项



Intel x86的内存管理硬件!

- 保护模式，地址32位，4G。



页部件把线性地址转换为物理地址的例子

Intel x86的内存管理硬件!

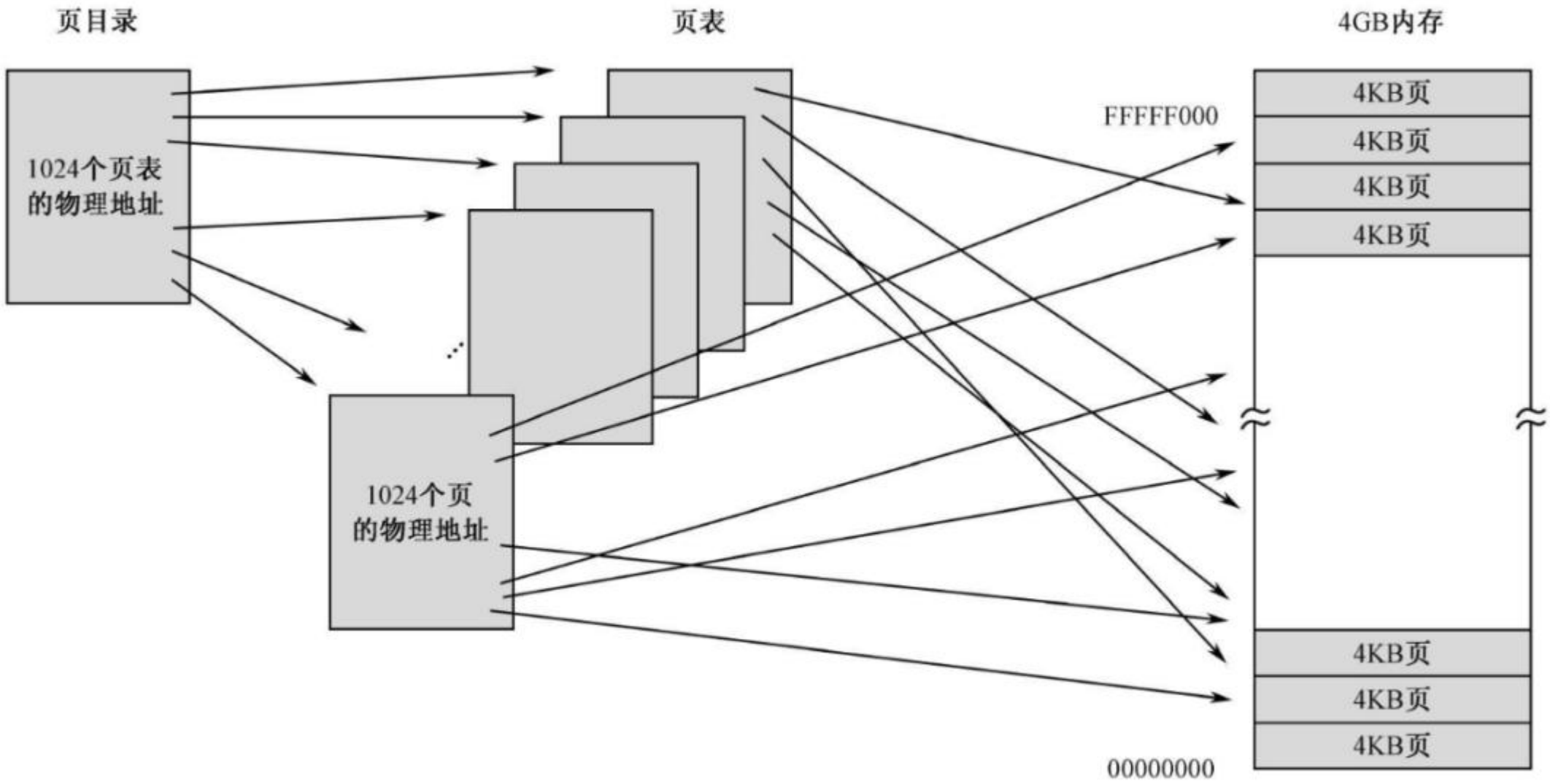
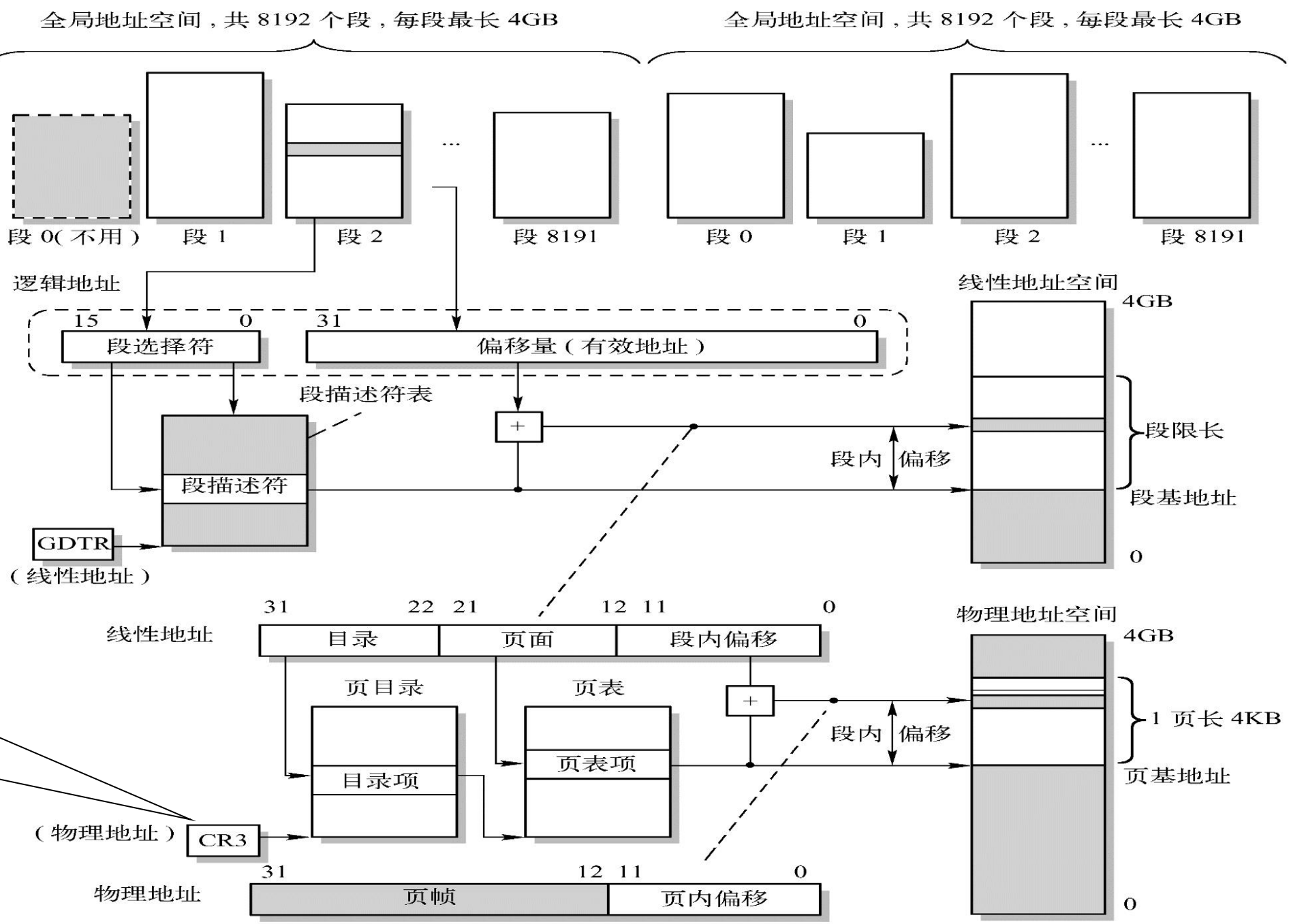


图16-6　页目录、页表和页的对应关系

X86地址变换



多个进程的段基址是否可以重叠？

内存管理总结

- 内存的根本目的 ⇒ 把程序放在内存并让其执行
- 程序执行需要重定位 ⇒ 编译、载入和运行三种定位时刻
- 运行时重定位最成熟 ⇒ 从逻辑地址到物理地址的翻译
- 内存如何管理 ⇒ 连续内存分配（分区）最直观
- 程序由若干段组成 ⇒ 以段为单位的内存分区策略 ⇒ 分段
- 分段对程序员自然，但会造成内存碎片⇒ 分页 ⇒ 段页结合
- 翻译、保护、内存分配是内存管理的三个核心词!

随堂讨论：1)描述一下段页结合的内存地址翻译过程



段号+偏移(cs:ip)

段号	基址	长度	保护
0	0x4000	0x0800	R
1	0x4800	0x1400	R/W
2	0xF000	0x1000	R/W
3	0x0000	0x3000	R

逻辑地址

页号 | 偏移

常称为线性地址，以示区别

物理地址

物理页号 | 偏移

页框号	保护
5	R
1	R/W
3	R/W
7	R

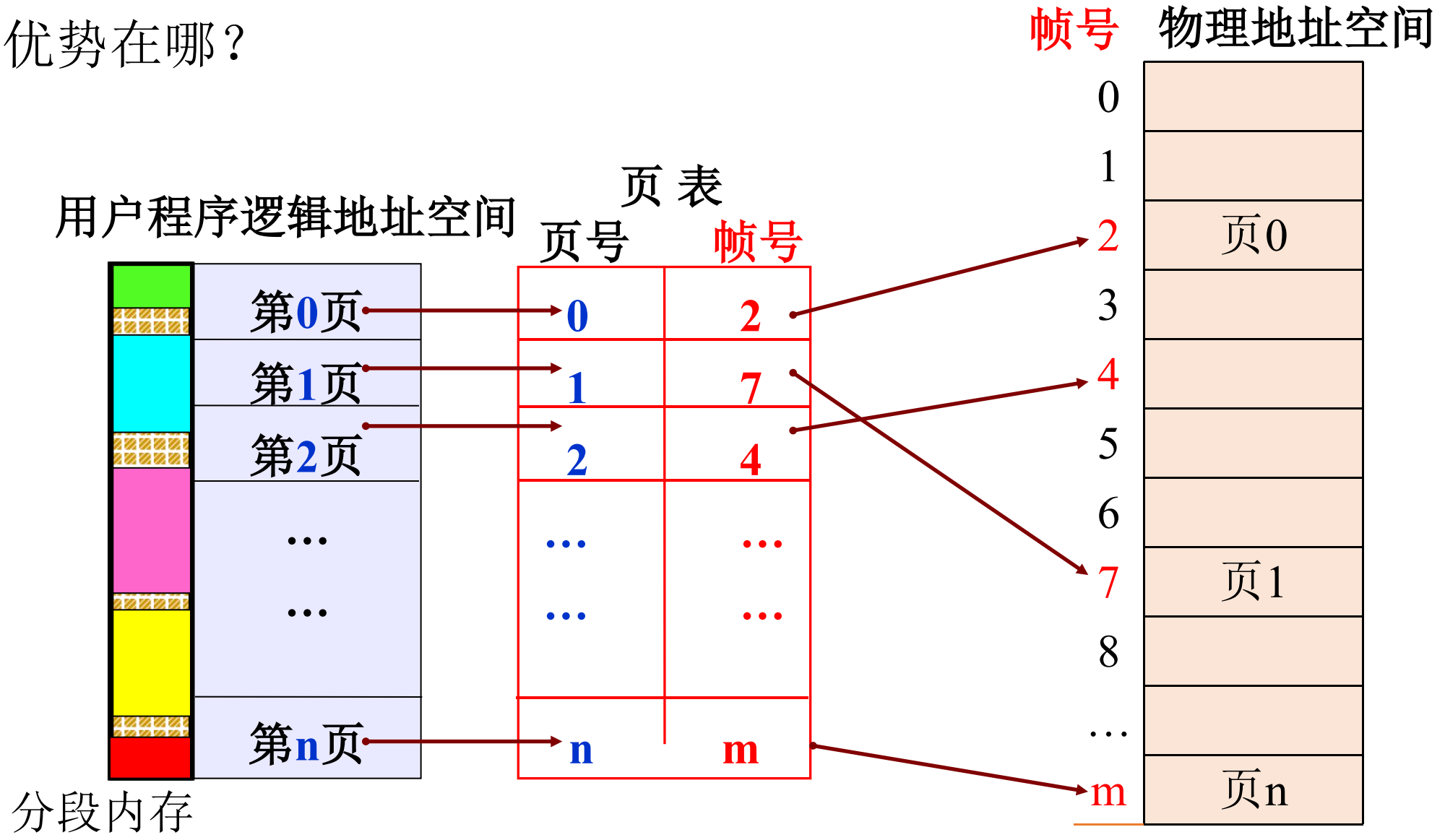


思考：翻译逻辑地址 0xc2f0、 0x0240 ？

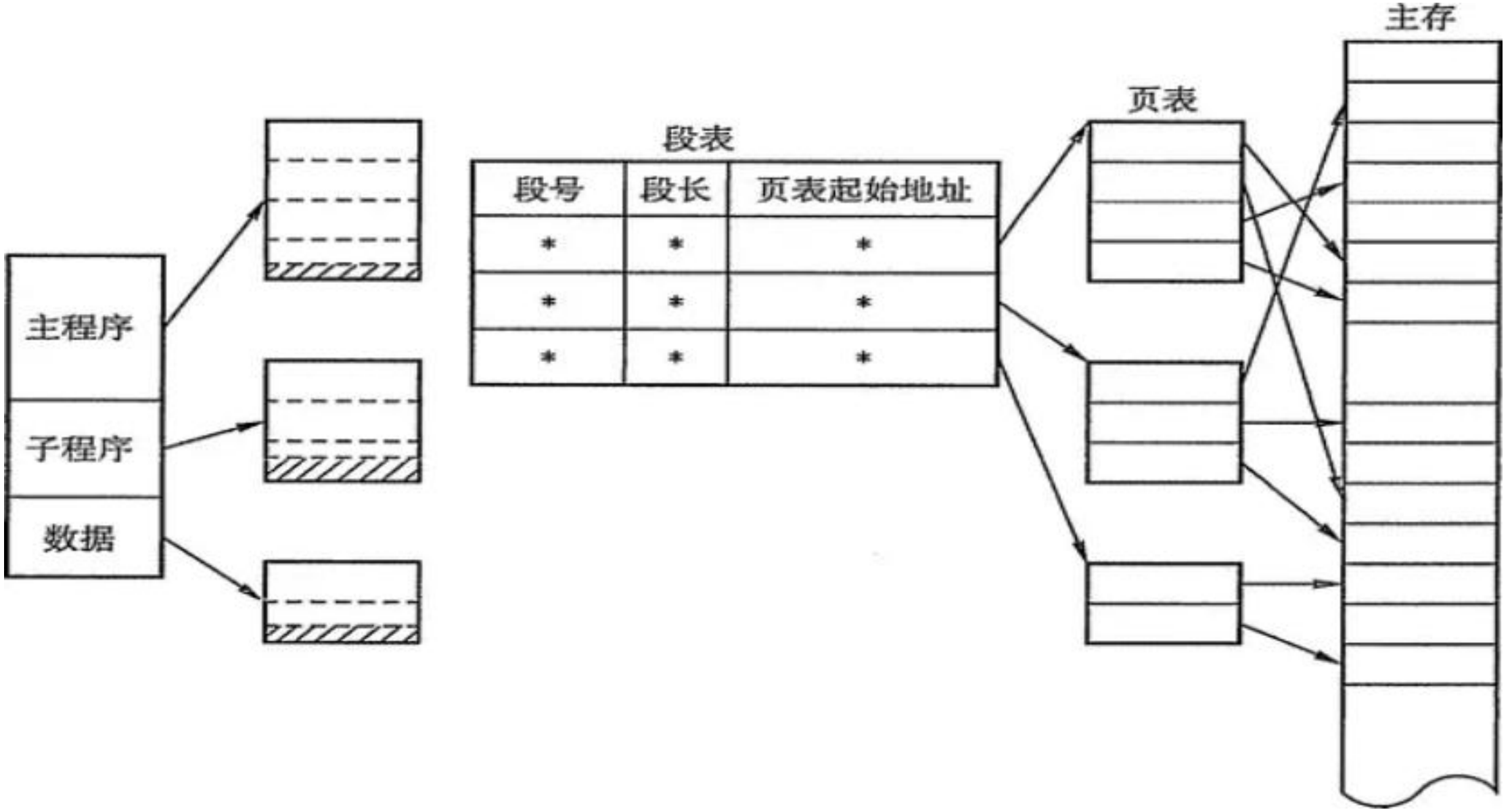
逻辑地址：15-14|13-0
线性地址：13-12|11-0

随堂讨论： 2)描述一下段页结合优势

- 如此映射优势在哪？

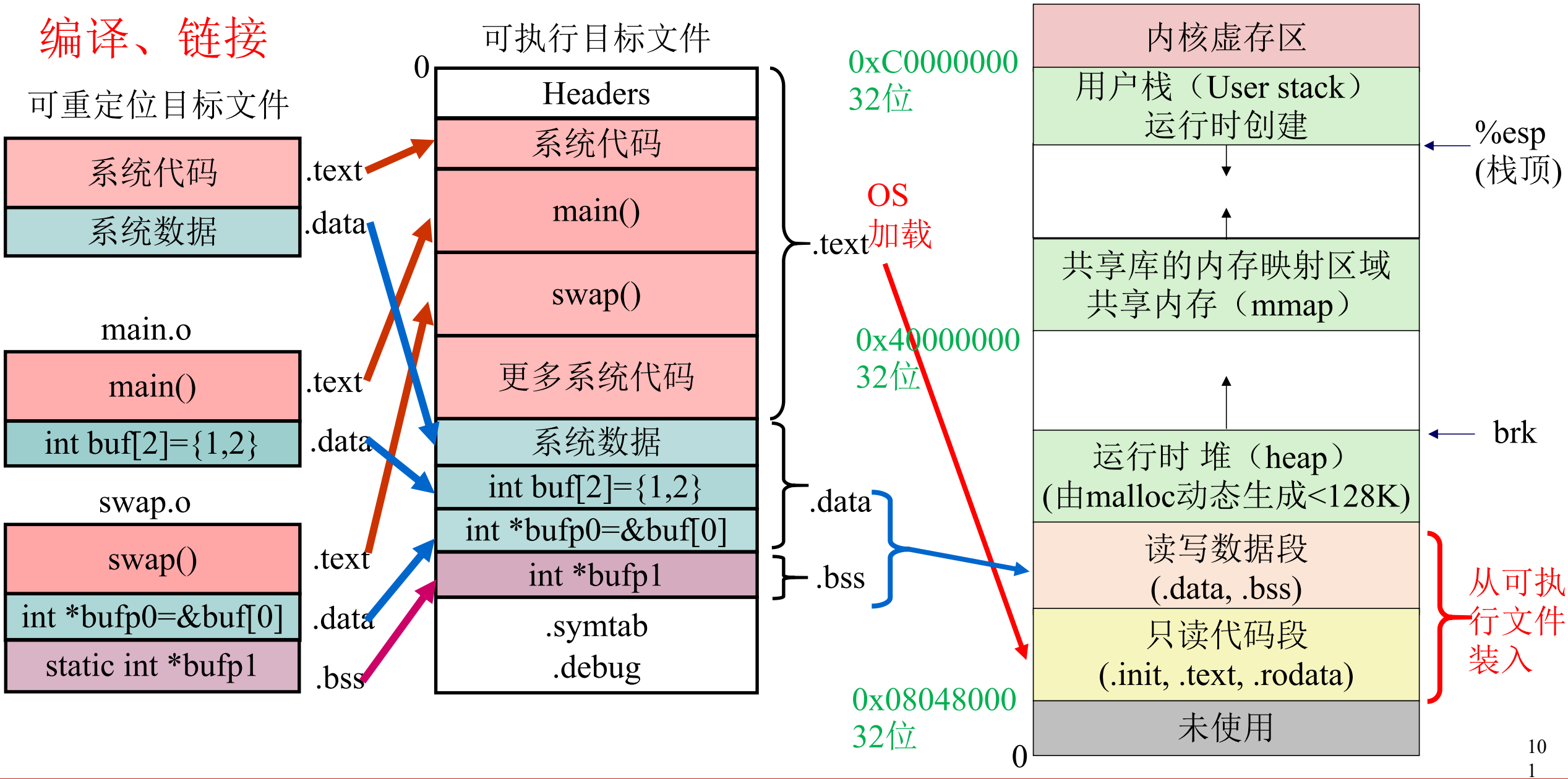


段页式管理总结



源程序、目标文件、执行程序、虚拟内存映像

编译、链接



随堂习题

- 例：某系统按字节编址，采用48位逻辑地址，页面大小为4KB，页表项大小为4B，假设采用纯页式存储，则要采用（）级页表，页内偏移量为（）位？

Hope you enjoyed the OS course!